

BloodHound

Prepared by: Noah Holloway, Micah Spencer

Spring 2026

Prepared for: Shane Slack, Tim Rossiter



Figure 1: Bloodhound Front View

Table of contents

[Abstract \(p.5\)](#)

[Background \(p.6\)](#)

[Proposed Problem \(p.10\)](#)

[Environmental & Ethical Considerations \(p.12\)](#)

[System Overview \(p.14\)](#)

[Troubleshooting Procedure \(p.30\)](#)

[Conclusion/ Recommendations \(p.52\)](#)

[Cost Analysis \(p.54\)](#)

[Engineering Log \(p.56\)](#)

[Appendix \(p.74\)](#)

Table of figures

- Figure 1: Bloodhound Front View (p.1)
- Figure 2: Bloodhound Frame & Hardware (p.8)
- Figure 2.1: Bloodhound Mounting Plate (p.14)
- Figure 2.2: Vehicle Actuators Block Diagram (p.18)
- Figure 2.3: Power Architecture Block Diagram (p.19)
- Figure 3: Universal Remote Controller (URC) (p.20)
- Figure 4: Master/ Main Board (p.21)
- Figure 4.1: Master Main Block Diagram (p.21)
- Figure 5: Slave Development Board (p.22)
- Figure 5.1: Slave Development Board Block Diagram (p.22)
- Figure 6: Piksi Console Display for Satellite Tracking (p.23)
- Figure 7: Hardware Setup for the Base Station Piksi (p.24)
- Figure 8: Broadcast Setup (p.25)
- Figure 9: UART Setup for both Base and Rover Piksi's (p.25)
- Figure 10: Program Setup for Rover (p.26)
- Figure 11: Program Setup to Disable extra NMEA Transmissions (p.27)
- Figure 12: Troubleshooting Flowchart; Power and Voltage Checks (p.30)
- Figure 13: Troubleshooting Flowchart; Communication Links (p.31)
- Figure 14: Troubleshooting Flowchart; Control Response Tests (p.32)
- Figure 15: Joystick Data Packet Findings from March 18 Engineering Log (p.73)
- Figure 16: Wiring Diagram Sketch for Battery Power (p.73)
- Figure 17: Wiring Diagram Sketch for Headlights & 12V Relays (p.74)
- Figure 18: Hardware Mounting Panel Layout (Slack 2026) (p.74)
- Figure 19: Universal Controller Schematic, Page 1 (Christensen 2024) (p.75)

Figure 20: Universal Controller Schematic, Page 2 (Christensen 2024) (p.75)

Figure 21: Universal Controller Schematic, Page 3 (Christensen 2024) (p.76)

Figure 22: Universal Controller Schematic, Page 4 (Christensen 2024) (p.76)

Figure 23: Universal Controller Schematic, Page 5 (Christensen 2024) (p.77)

Figure 24: Universal Master/ Main Board Schematic (Slack 2025) (p.78)

Figure 25: PIC Development/ Slave Board Schematic, Page 1 (Slack 2025) (p.79)

Figure 26: PIC Development/ Slave Board Schematic, Page 2 (Slack 2025) (p.79)

Figure 27: PIC Development/ Slave Board Schematic, Page 3 (Slack 2025) (p.80)

Figure 28: PIC Development/ Slave Board Schematic, Page 4 (Slack 2025) (p.80)

Figure 29: PIC Development/ Slave Board Schematic, Page 5 (Slack 2025) (p.81)

Figure 30: PIC Development/ Slave Board Schematic, Page 6 (Slack 2025) (p.81)

Abstract

The Bloodhound Remotely Controlled Electric Vehicle (RCEV) is a capstone engineering project undertaken at Idaho State University's Robotics and Communications Engineering Technology (RCET) program. The project repurposes a Kawasaki 450 all terrain vehicle frame as the mechanical foundation for a teleoperated electric platform capable of operating in environments hazardous to human personnel. The vehicle is powered by a 52V LiFePO₄ battery pack and driven by two 48V brushless hub motors providing front wheel drive, with steering controlled through a DOCYKE power steering servo mounted to the steering column. Directional commands and vehicle control are transmitted from a custom Universal Remote Controller (URC), built around a PIC16LF1789 44-pin microcontroller, to the vehicle via a wireless UART link at 115,200 bps. Onboard, a PIC16F1788 based Master Main Board decodes the incoming control data and distributes it to a Slave Development Board over a 100 kHz I2C bus. The Slave board translates joystick position data into PWM signals that drive the DOCYKE servo and interfaces with the dual brushless speed controllers to produce forward, reverse, and stopped drive states. Custom firmware developed in MPLAB X handles all timing, communication, and safety watchdog functionality, including an I2C communication timeout that returns the vehicle to a safe stop state when the control link is lost. The architecture is deliberately modular, with future expansion planned for front and backup cameras, headlight control, an accelerometer for positional awareness, LiDAR, thermal and night vision imaging, and environmental chemical sensors for CO and CO₂ detection. This report documents the design, hardware implementation, firmware architecture, and troubleshooting procedures developed over the project's eight week development period.

Background

Origins of the Project

The Bloodhound RCEV began as an opportunity to give an otherwise retired piece of hardware a meaningful second life. An existing Kawasaki 450 ATV frame, complete with its suspension geometry, wheels, and mechanical steering column, was available as the starting point, presenting a practical and cost effective chassis for a remotely operated vehicle platform. Rather than designing or fabricating a frame from scratch, the decision was made to strip the internal combustion drivetrain and repurpose the vehicle as an electric, remote controlled platform. This approach aligned well with the dual goals of sustainable engineering practice and managing build cost within a student project budget.

The broader motivation came from the growing demand for unmanned ground vehicles (UGVs) capable of operating in environments that are dangerous, physically inaccessible, or otherwise unsuitable for direct human presence. Applications such as hazardous material reconnaissance, post disaster search and rescue, and remote environmental monitoring all benefit from a rugged, low cost platform that does not require a purpose built chassis. An ATV frame, with its proven off-road suspension, wide stance, and robust construction, is a natural fit for this type of work.

RCET Program Context

The Bloodhound is developed within the RCET capstone course at Idaho State University, which requires students to design, build, and document a complete embedded systems project from concept to functional prototype within an eight week timeframe. The

project draws on coursework in microcontroller programming, digital communications, power electronics, and embedded system design. The three board architecture (Universal Remote Controller, Master Main Board, and Slave Development Board) was developed iteratively, with prior student work in the program establishing the URC platform (PIC16LF1789, fall 2024) and the general Master Board template (PIC16F1788, spring 2025), on which this project builds. Credit for the Universal Remote Controller board, as well as the accompanying foundational code for it, the Master Main Board, and the Slave Development board goes to Zac Christensen, a former graduate of the ISU RCET program (See References for full credit). Only minor modifications to the existing code were needed to make the Bloodhound RCEV perform the basic operations of driving, steering, and braking.

Prior Work and Existing Hardware

The Universal Remote Controller PCB and its firmware were initially developed in the Fall 2024 semester. The URC uses a PIC16LF1789 in a 44-pin TQFP package and provides three analog joystick inputs, six button inputs, bumper switches, a robot address DIP switch, and a UART communication interface. The Master Main Board PCB was developed in the Spring 2025 semester as a template platform for the PIC16F1788, featuring voltage regulation, I2C ports with 4.7 k Ω pull-up resistors and an I2C level shifting circuit, a UART interface, analog and digital input conditioning, an 8 position address DIP switch, and debug LED indicators. The Slave Development Board, also built around the PIC16F1788, serves as the actuator facing controller and was the primary firmware development target for this capstone.

The physical ATV components; the Kawasaki 450 frame, suspension, wheels, DOCYKE power steering servo, dual 48V brushless hub motors, dual 2 mode brushless speed controllers, 52V LiFePO4 battery pack with BMS, and the 30–60V to 24V DC/DC converter were sourced

and assembled as the vehicle platform prior to the start of the capstone design phase. This meant the firmware and electronic architecture had to accommodate fixed hardware interfaces rather than being designed from the ground up.



Figure 2: Bloodhound Frame & Hardware

Key Engineering Challenges Encountered

Several non-trivial problems were identified and resolved during early stages of the system. The I2C level shifting circuit on the Master Main Board relies on a pair of N-channel MOSFETs (Q1, Q2) to allow the 3.3V logic of the PIC16F1788 to communicate with 5V tolerant peripherals. These MOSFETs were found to be unpopulated on the initial board, causing SCL and SDA lines to read 0V and making I2C communication entirely non functional. Installing these transistors resolved the issue and restored normal I2C bus operation.

A second significant issue involved Timer 2 configuration on the Slave board. An incorrect T2CON value of 0x00 produced a timer interrupt period of approximately 62 μ s rather than the intended 4 ms, which caused the I2C communication timeout counter to reach its threshold almost immediately after power on, placing the system in a permanent timeout state before any valid I2C data could arrive. The correct configuration of T2CON = 0x79 (prescaler 1:4, postscaler 1:16) with PR2 = 249, produces the intended 4 ms interrupt period and allows the timeout mechanism to function as designed.

Proposed Problem

The Bloodhound RCEV project addresses the need for a safe, versatile, and remotely operable ground vehicle capable of entering hazardous or difficult to access environments in place of human operators. The core problem this project solves is human risk exposure in dangerous scenarios such as environments with toxic gases, structural instability, poor visibility, or other conditions where sending a person would be unsafe or impractical. By enabling remote teleoperation over a wireless communication link, the Bloodhound allows an operator at a safe distance to maneuver the vehicle, observe its surroundings through onboard cameras, and eventually monitor environmental conditions through chemical and atmospheric sensors.

The primary challenge is integrating the existing hardware (speed controllers, power steering, and battery) with a reliable bidirectional communication system that can transmit control commands to the vehicle while simultaneously streaming sensor and camera data back to the operator, all within an 8 week time frame.

Additional constraints include working within the limitations of the brushless speed controller's existing signal interfaces, managing power distribution safely across drive and electronics systems, and building a modular architecture that allows future expansion into LiDAR, hazardous chemical sensing, thermal/night vision, headlight, etc. all without requiring a full system redesign.

Goals:

- **Remote Control:** Establish reliable bidirectional wireless communication between the universal remote and the Bloodhound, ensuring proper shutdown should the signal become interrupted or lost.

- **Remote Drive/ Steering Control:** Interface the DOCYE power steering servo and brushless motors with the remote control system to translate joystick input into precise directional commands on the steering column and wheels.
- **Power System Integration:** Safely configure the 52V battery and DC/DC converter to power both the drive motors and all onboard electronics (communication, cameras, and lighting).
- **Onboard Camera and Headlight Control:** Mount and integrate a front camera and backup camera with automatic switching when reverse is engaged, streaming live video to the operator, along with remotely controllable headlights.
- **Positional Awareness via Accelerometer:** Integrate an accelerometer to provide real time orientation and motion data independent of drive commands, establishing the foundation for future autonomous navigation features.

Environmental & Ethical Considerations

Benefits to Society

Remote vehicles like the Bloodhound shine in dangerous or hard to reach spots. First responders and hazmat crews can scout chemical hazards, like high CO or CO₂ levels, without putting people in harm's way. Building it from an existing ATV frame combined with off the shelf motors and cheap microcontrollers keeps costs low and cuts down on new or unnecessary electronic waste. This reuse approach feels like smart, sustainable engineering. Plus, projects like this give RCET students hands on experience in building embedded systems, helping meet the growing demand for robotics skills and competency.

Adverse Impacts and Ethical Concerns

With that being said, a 450cc ATV frame has some serious energy. If firmware loses communication, the vehicle could keep going and cause real damage. The current I2C timeout helps, but software alone isn't enough. We need hardware failsafes like a physical kill switch. Engineers have a clear duty to put public safety first, and student projects should practice that same standard.

Privacy is another big issue. The cameras and thermal imaging can watch people without their knowledge or consent. Even if we're only using it for obstacle detection now, the same hardware could easily shift to unauthorized surveillance. As these platforms get cheaper, this concern becomes more and more real.

Environmentally, switching from gas to electric eliminates tailpipe emissions and noise pollution. However, producing LiFePO₄ batteries still carries costs, where lithium mining is water

intensive and can disrupt local ecosystems and communities. Responsible design means looking at the full lifecycle, not just the cleaner ride.

Opposing Viewpoints

Some argue these robots should be used widely in search and rescue and public safety because they save lives and the benefits outweigh privacy worries. Others warn that normalizing remote surveillance tools breaks trust and risks misuse by governments or private companies. Both sides have valid points/ concerns, which is why ethical thinking needs to start at the very beginning of any project.

Summary

The Bloodhound delivers real value: safer hazard assessment, green reuse of hardware, and effective student learning. But it also comes with responsibilities. We need better physical safety backups, clear policies around sensor use to protect privacy, and honest accounting of battery supply chain impacts. Doing ethical engineering right means facing these issues head on, not tacking them on later. This project is not just a cool remote ATV, it is practice for building technology that serves a real purpose in helping people.

System Overview

(See Appendix for Schematics and Github Repository)

System Description

The Bloodhound RCEV is a hierarchical, multi board embedded control system layered on top of a converted electric ATV platform. At the highest level, the operator interacts with a handheld Universal Remote Controller, which captures joystick and button inputs and transmits them in a structured packet format to the vehicle. Onboard the vehicle, a Master Main Board receives and decodes those packets, then distributes control data over a local I2C bus to one or more Slave boards, each responsible for driving a specific set of actuators. In the current configuration, a single Slave Development Board handles steering via the DOCYKE power steering servo and drives via the two brushless motor speed controllers. The 52V LiFePO4 battery powers the drive system directly and supplies the DC/DC converter, which produces a regulated 24V rail for the servo and onboard electronics.

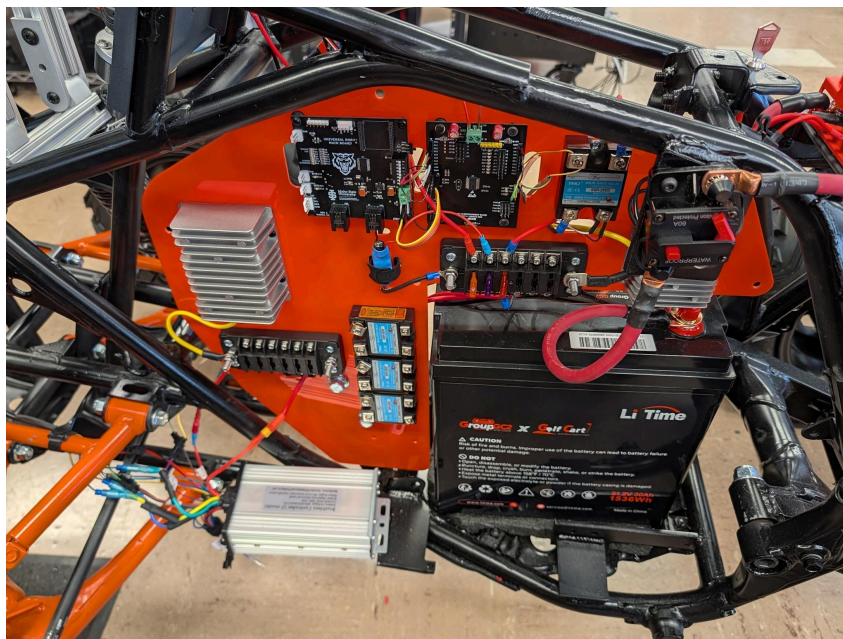


Figure 2.1: Bloodhound Mounting Plate

The system is designed with modularity as a core principle. Each slave board operates independently on the I2C bus with its own address, selected via DIP switch. Adding a second slave, for example, to control headlights, a camera switch relay, or future sensor payloads, requires only assigning a different address and adding the corresponding I2C transaction in the master firmware's SEND_I2C routine, without restructuring the existing communication chain. Safety is embedded at multiple levels. The URC firmware monitors for packet integrity using a handshake byte (0x24) and a robot address byte before accepting any data. The Master firmware tracks time between valid UART receptions using Timer 2; if no valid packet arrives within approximately 0.5 seconds, the CONNECT_TIMEOUT routine is invoked. On the Slave side, Timer 1 independently monitors the time between valid I2C data packets; if the I2C link goes silent for more than approximately 260 ms (two Timer 1 overflow cycles), the I2C_COM_TIMEOUT routine fires and commands all motor outputs to the neutral/stop state, preventing uncontrolled vehicle motion in the event of any communication failure.

Communication Architecture

The communication chain flows as follows:

- 1.) The URC captures analog joystick data (three joysticks, each providing up/down and left/right analog values) and digital button states on every processing cycle. It organizes this data into three structured UART packets, each six bytes long: a sync byte (0x24), a robot address byte, a command identifier byte, and three data bytes. The three packet types are the Joystick 1 packet (command 0x4A), the Joystick 2 packet (command 0x6A), and the Button packet (command 0x42). These are transmitted sequentially over UART at a baud rate of 115,200 bps, 8-N-1 framing, to the Master board via a wireless connection.

- 2.) The Master board's UART receive interrupt (triggered by PIR1,5) processes incoming bytes one at a time through the UART_RX routine in URC_RX_Handler.inc. BYTE_COUNT tracks position within the packet; the first byte is checked against the handshake constant, the second against the stored robot address in ROBOT_ADDR (which mirrors the Port B DIP switch state), the third is saved as the command type, and bytes four through six are saved to MEM_SAVE_1 through MEM_SAVE_3. On receipt of the sixth byte, the NEW_RX_DATA flag (bit 0) is set, and CONNECT_CHK_COUNT is cleared to signal a healthy connection. The main loop then calls DECODE_URC, which routes the three data bytes to their respective joystick or button registers (JOY1_UD, JOY1_LR, JOY2_UD, JOY2_LR, JOY3_UD, JOY3_LR, BUTTON_STATUS_1, BUTTON_STATUS_2) based on the command byte.

- 3.) Every 96 ms (determined by the Timer 2 interrupt incrementing I2C_COM_DELAY to a threshold of 3, with each interrupt period approximately 32 ms), the NEW_RX_DATA flag bit 1 is set, triggering the SEND_I2C routine. This routine performs two sequential I2C write transactions: one to the Drive Control board (address 0x10 write / 0x11 read) carrying Joystick 1 and Joystick 2 data, and one to the Peripheral Control board (address 0x20 write / 0x21 read) carrying Joystick 3 data and button status bytes. Each transaction consists of a START condition, the 7-bit slave address plus write bit, four data bytes, ACK checks after each byte, and a STOP condition. If any byte goes unacknowledged, BAD_COND generates a STOP and returns 0xFF to the caller.

- 4.) On the Slave board, the I2C MSSP peripheral operates in 7-bit slave mode with SSPADD = 0x20, matching the peripheral control write address assigned by the master. Each received byte triggers an SSP interrupt (PIR1,3), which calls the I2C_RECEIVE routine. This routine determines whether each received byte is the address or a data

byte (using the D_NOT_A bit in SSPSTAT), saves data bytes sequentially into I2C_RX_TEMP_1 through I2C_RX_TEMP_4, and sets the I2C_RX_COMPLETE flag after the fourth data byte arrives. It also clears the Timer 1 counter registers on each successful fourth-byte receipt, resetting the I2C communication watchdog. Clock stretching (CKP bit in SSPCON1) is used to hold the bus while each byte is processed.

- 5.) Once I2C_RX_COMPLETE is set, the main loop calls UPDATE_DRIVE_STATUS, which saves the four temporary registers into the working joystick registers (JOY1_UD, JOY1_LR, JOY2_UD, JOY2_LR), then calls DETERMINE_DRIVE_STATE to compare each joystick axis against upper (0xC8) and lower (0x28) thresholds, builds a CURRENT_DRIVE_STATE byte encoding left and right motor directions in separate nibbles, and finally calls SET_DRIVE_STATE to translate that state byte into the appropriate PWM on-time count values for the right and left motor outputs and the actuator motor output.

PWM Output and Actuator Control

The Slave board generates PWM signals for the servo and speed controllers through a software timer approach using Timer 2. Timer 2 is configured with T2CON = 0x79 (prescaler 1:4, postscaler 1:16) and PR2 = 249, producing a 4 ms interrupt period. Two cascaded counters, T20MS_COUNT_1 and T20MS_COUNT_2, divide this down to a 20 ms cycle period, matching the standard servo PWM frame. At the start of each 20 ms cycle, all three output pins (PORTA,0 for right motor, PORTA,1 for left motor, and PORTA,4 for the DOCYKE servo) are set high, and the corresponding count registers are loaded from their respective STATUS registers. On each subsequent Timer 2 interrupt, the count registers are decremented; when a count reaches zero, the corresponding output pin is cleared. This produces pulse widths of 1.8 ms (left

turn / full left), 1.5 ms (stop / center), and 1.2 ms (right turn / full right). These values are compatible with standard servo protocol.

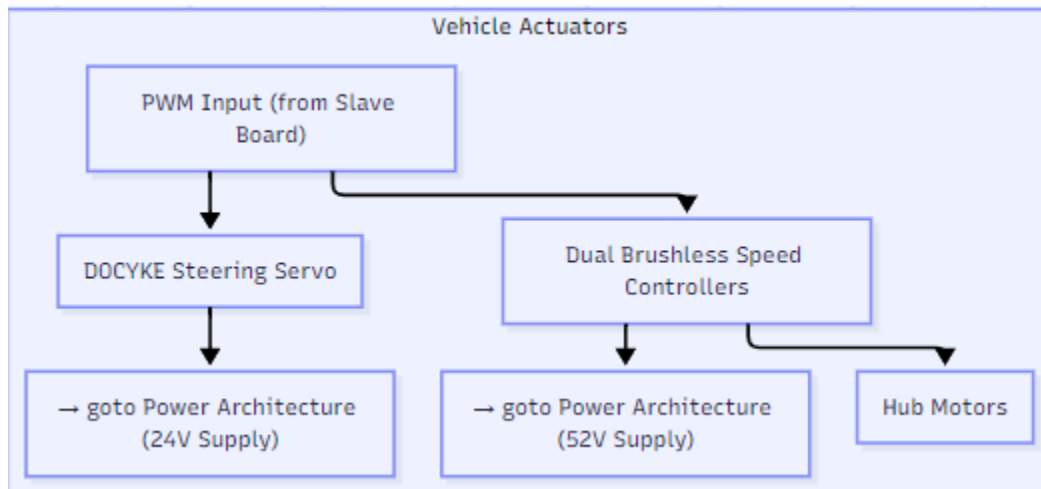


Figure 2.2: Vehicle Actuators Block Diagram

The DOCYKE power steering servo (model S350/S550) operates on 16–24V supply voltage, accepts a PWM signal at 50 Hz with pulse widths from 1 ms- 2 ms, and requires a signal voltage of 3.3–5V — all within the parameters produced by the Slave board. The servo provides a rated torque of 12–18 Nm and a servo mode travel speed of 0.16–0.24 seconds per 60 degrees, sufficient for responsive steering on the ATV platform.

Power Architecture

The 52V LiFePO₄ battery (nominal 51.2V, full charge approximately 58.4V, minimum 44.8V) is protected by the onboard Battery Management System (BMS), which monitors cell balance, overcurrent, and temperature. High-voltage power from the BMS output feeds the two brushless speed controllers directly. A separate DC/DC converter accepts 30–60V input and produces a regulated 24V, 10A output for the DOCYKE servo and for the onboard logic boards. The PIC microcontrollers and their associated circuitry operate at 3.3V and 5V, provided by linear regulators on the Master and Slave boards derived from the 24V rail. The Master Main Board uses the 3.3V rail, with filtering capacitors on both input and output.

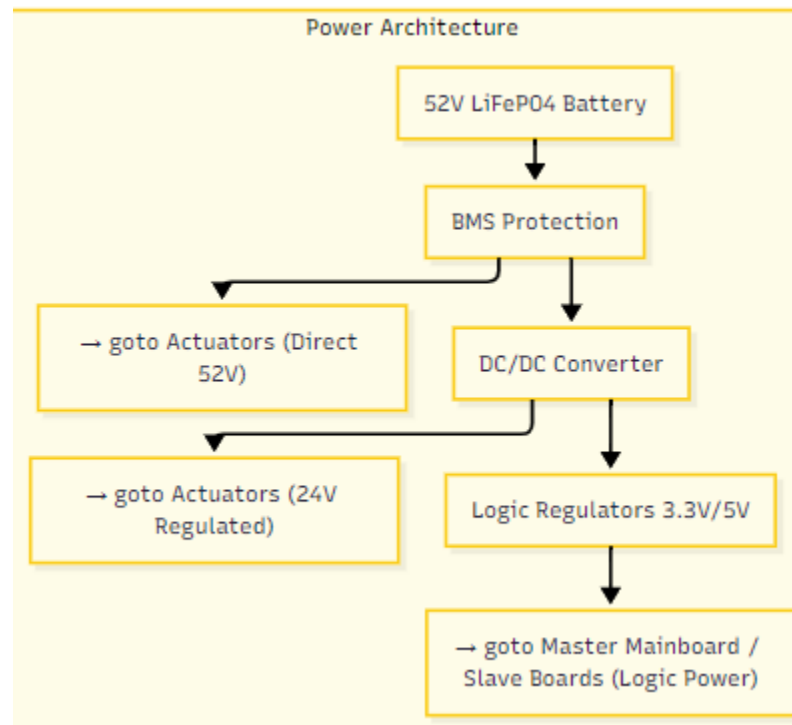


Figure 2.3: Power Architecture Block Diagram

Hardware Platforms

Universal Remote Controller (URC): Built around the PIC16LF1789 in a 44-pin TQFP package. Input peripherals include three dual-axis analog joysticks (U2–U4), six momentary

push buttons, two bumper switches, and an 8-position DIP switch for robot address selection. Joystick 1 and 2 axes are wired to Port A and Port B analog pins (RA0–RA5, RB0–RB2 for joystick U/D and L/R signals) with pull-up and pull-down resistors and 0.1 μF filter capacitors at each input. Button inputs are connected to Port C and Port E via 1 k Ω pull-down resistors. UART TX and RX are available on RC6/TX and RC7/RX. An ICSP programming header (J1) allows firmware updates via PICKit 4.

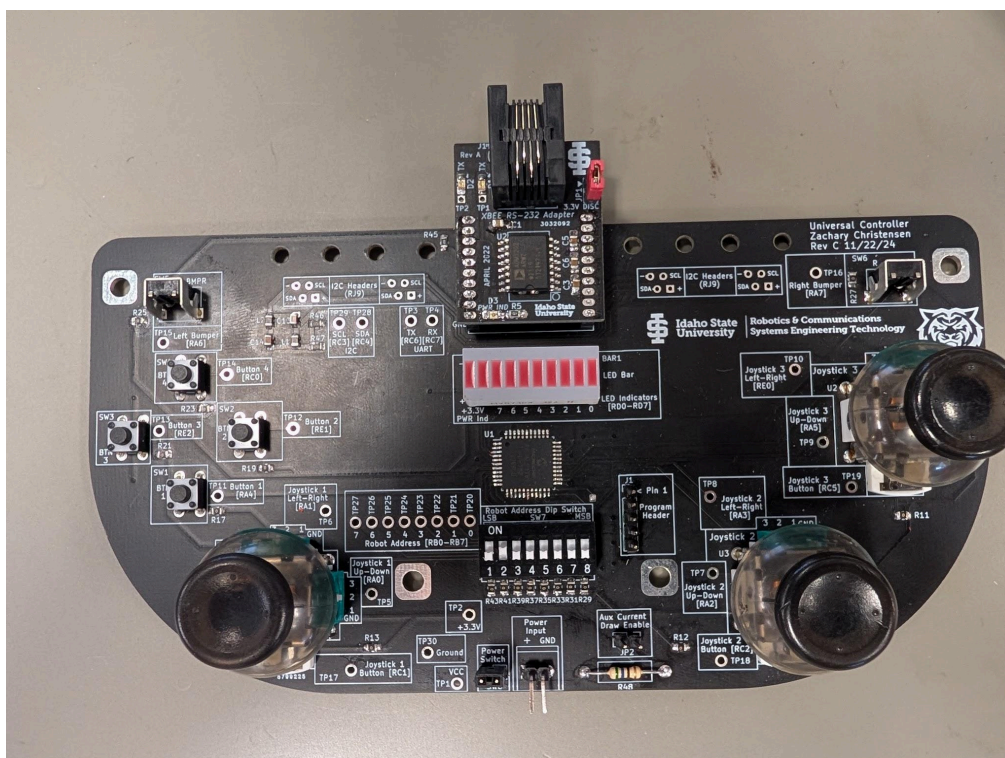


Figure 3: Universal Remote Controller (URC)

Master/ Main Board: Built around the PIC16F1788 in a 28-pin DIP package. The board includes an 8-bit address DIP switch driving Port B (used by the URC receive handler to verify the robot address byte), UART receive on the RC7/RX pin, I2C SCL and SDA on RC3 and RC4 with 4.7 k Ω pull-up resistors to 5V and the Q1/Q2 N-channel MOSFET level-shifting circuit, analog input conditioning on RA4, digital input conditioning with MMBT2904 transistors on RA4 and RA5, eight debug LEDs (D4–D11) on Port A driven through 470 Ω resistors, an ICSP

header, and mounting holes. Power is supplied from the 24V rail through the ASM8117-3.0 to produce 3.3V logic supply.

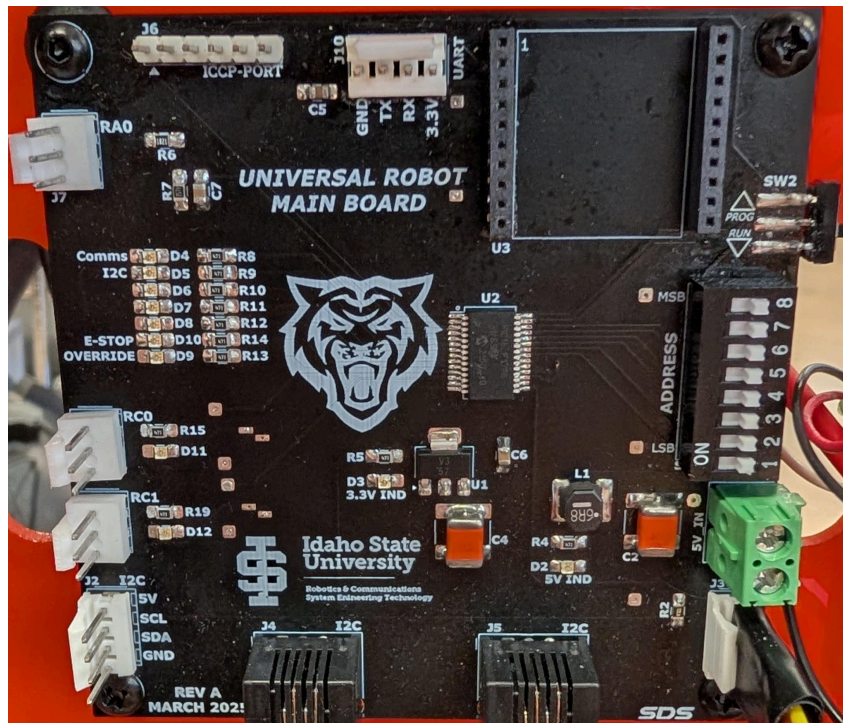


Figure 4: Master/ Main Board

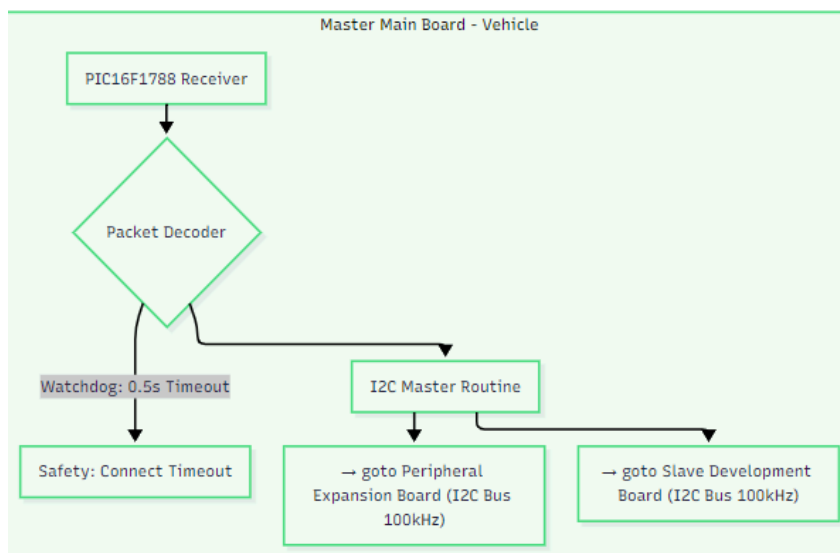
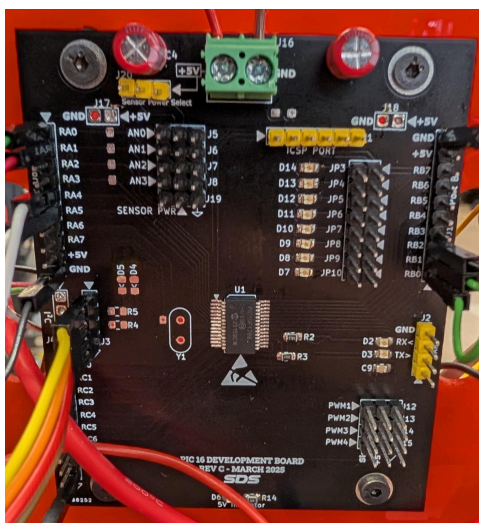


Figure 4.1: Master Main Block Diagram

Slave Development Board: Built around the PIC16F1788 in a 28-pin package. The board schematic (Rev A, drawn by Shane Slack, March 2025) shows a modular hierarchy — Power, IO Ports, Microcontroller, Communications, and Debug sub-sheets. The PIC16F1788 uses its internal 16 MHz oscillator (OSCCON = 0x78, no external crystal). An ICSP port (J1) is provided. Port A outputs (RA0, RA1, RA2) carry the PWM signals to the right motor controller, left motor controller, and DOCYKE servo respectively, via the Port A header J9. Port C pins RC3 (SCL) and RC4 (SDA) carry the I2C bus. Debug LEDs D4 and D5 on LATA,0 and LATA,1 provide visual confirmation of I2C send events and valid UART address receipt.



Block Diagram Description

At the top level, the operator holds the URC, which accepts joystick and button inputs and produces UART data packets. These packets travel over a wired UART link (115,200 bps, 8-N-1) to the Master Main Board on the vehicle. The Master decodes incoming data and, on a 96 ms cadence, transmits four-byte I2C write transactions to each Slave board over the onboard I2C bus (100 kHz, pulled to 5V). The Slave Development Board receives the I2C packets, interprets joystick data against threshold values, and drives three PWM outputs: one to the right-side brushless speed controller, one to the left-side brushless speed controller, and one to the DOCYKE power steering servo. Both speed controllers power the 48V brushless hub motors. The 52V battery feeds the speed controllers and the DC/DC converter; the DC/DC converter provides 24V to the servo and the onboard electronics.

Two watchdog timers operate independently. On the Master, Timer 2 counts elapsed time since the last valid UART packet; if the count surpasses approximately 0.5 seconds without a valid reception, `CONNECT_TIMEOUT` fires. On the Slave, Timer 1 counts elapsed time since the last valid I2C packet fourth byte; if that counter overflows twice without a reset (approximately 260 ms of I2C silence), `I2C_COM_TIMEOUT` fires and drives all outputs to neutral.

Planned Expansion

The current system establishes the communication backbone and actuator interface necessary for remote driving. The following subsystems are defined as future additions within the project scope and will be documented as they are implemented: Headlight control will be implemented as a relay driver output on the Slave board, toggled by the button status byte from the URC. The backup camera will be routed through an analog video switch, with the switch control signal tied to the reverse bit in the drive state logic so that it activates automatically when

reverse is engaged; both the front camera and backup camera feeds will be streamed to a display at the operator station. An IMU/accelerometer (likely I2C-addressed) will be added to the Slave or a dedicated second Slave board to provide real-time orientation and motion data independent of drive commands, establishing the telemetry and awareness foundation for eventual autonomous navigation. LiDAR, thermal imaging, night vision, and environmental chemical sensors (CO, CO₂) are identified as long term expansion targets, each requiring dedicated interface boards and either additional Slave boards or a direct connection to the Master board's unused analog and digital input channels.

GPS RTK

Development of an Real-Time Kinematic (RTK) module for the BloodHound was started, but more work is required to become operational. A Swift Navigation PIKSI v2.3.1 kit was used as the RTK module. Two of these modules are required for 2-cm precision; One configured as a base station, and one to be connected to the BloodHound as a rover. Without the 2 modules, the gps will have a standard precision of 1-meter, instead of 2-cm.

These modules require at least 5 satellites to be connected with a signal-to-noise ratio above 33 dB-Hz in order for the position to be precise. For precision, the manufacturers recommended that both the base station and the rover both be outdoors. These modules also need a Piksi Console (software developed by Swift Navigation) that is up-to-date to program and verify the position. All documentation and the Piksi Console Installer are located in the GitHub, within the GPS Folder.

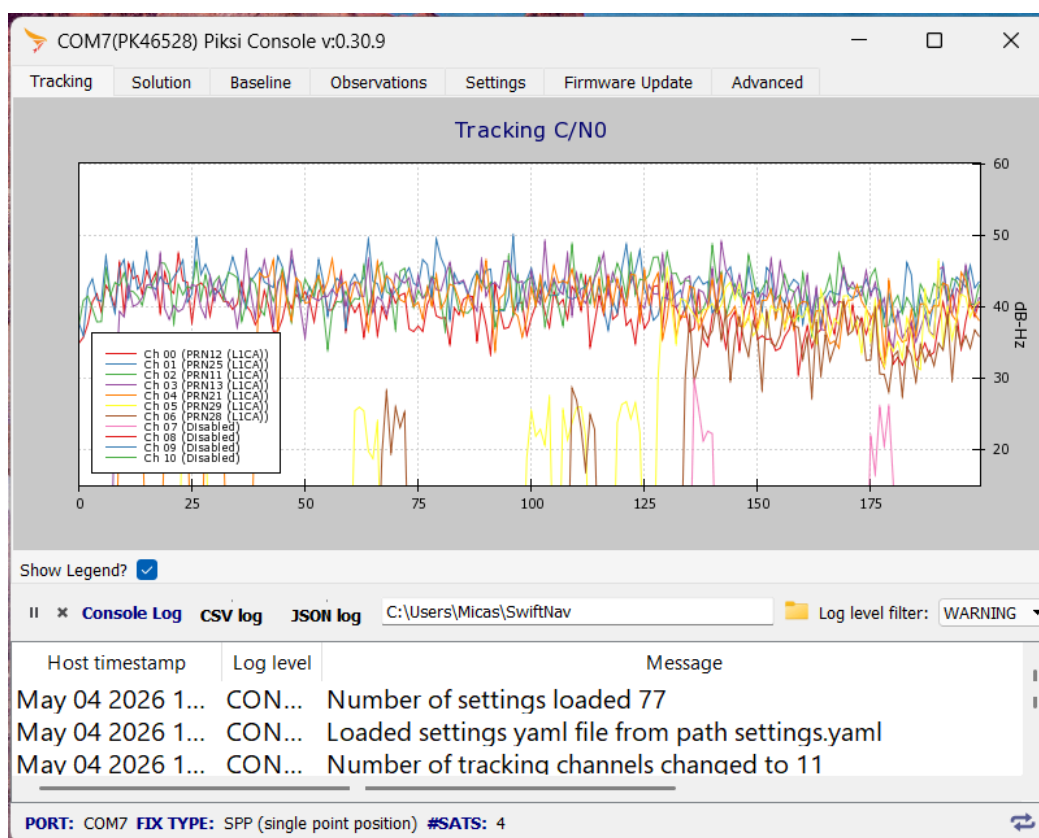


Figure 6: Piksi Console Display for Satellite Tracking

RTK Setup

To set up one of the modules as a base station, it needs to be connected to a static location. A tripod was used in this instance. The GPS Satellite antenna has to be unobscured, and above the Piksi Module as high as possible. The radio needs to be connected to UARTA, and the Piksi needs to be connected to a power source. The base station itself can be connected to a battery, but it is able to be connected to a computer with the Piksi Console.



Figure 7: Hardware Setup for the Base Station Piksi

The coordinates (Latitude, Longitude, and Altitude) of the base station needs to be programmed into the Piksi, the Piksi needs to be set up as a base broadcast, and the Piksi needs to be configured to transmit and receive data across UARTA (by placing a sbp mask of 64). Doing all of this will enable the Piksi to operate as a base station.

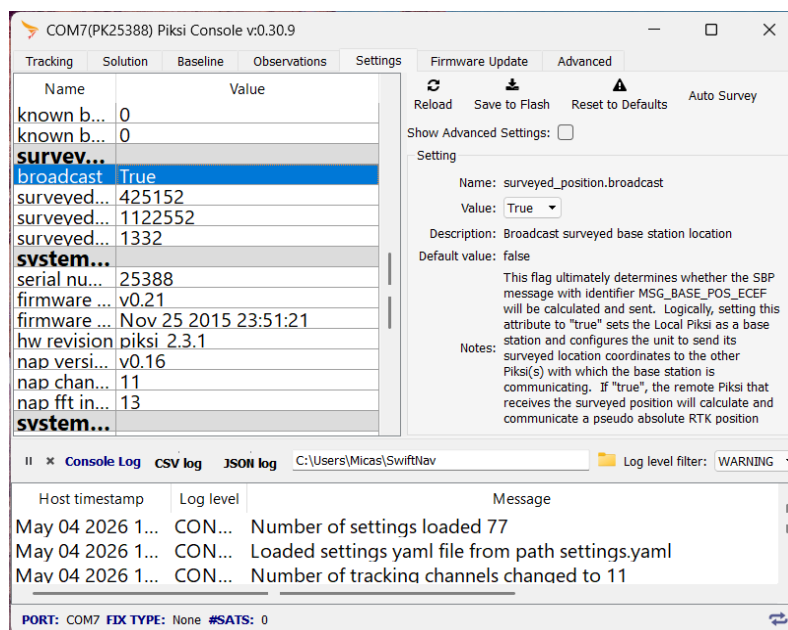


Figure 8: Broadcast Setup

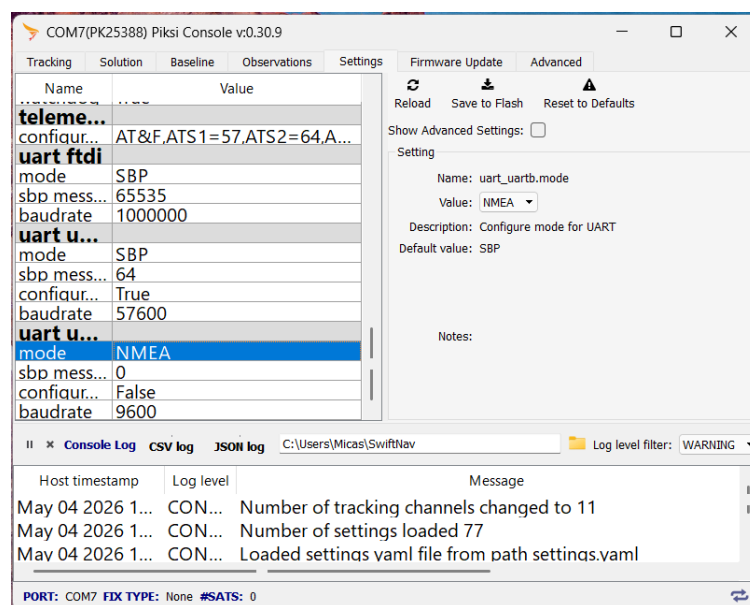


Figure 9: UART Setup for both Base and Rover Piksi's

To hook up the rover, the process is almost identical to the base station. The radio needs to be connected to UARTA, A data line needs to be connected to UARTB, and the GPS Antenna needs to be as high as possible and away from the Piksi. Power for the Piksi can either be connected through the USB cord, or through UARTB.

To program the rover, you need to disable the broadcast function. UARTB needs to be configured to NMEA transmission, and the baudrate has to run at 9600. UARTA needs to be configured to broadcast and receive data (with an sbp mask of 64). As of right now, the other NMEA transmissions need to be disabled, but further testing will be needed to verify this.

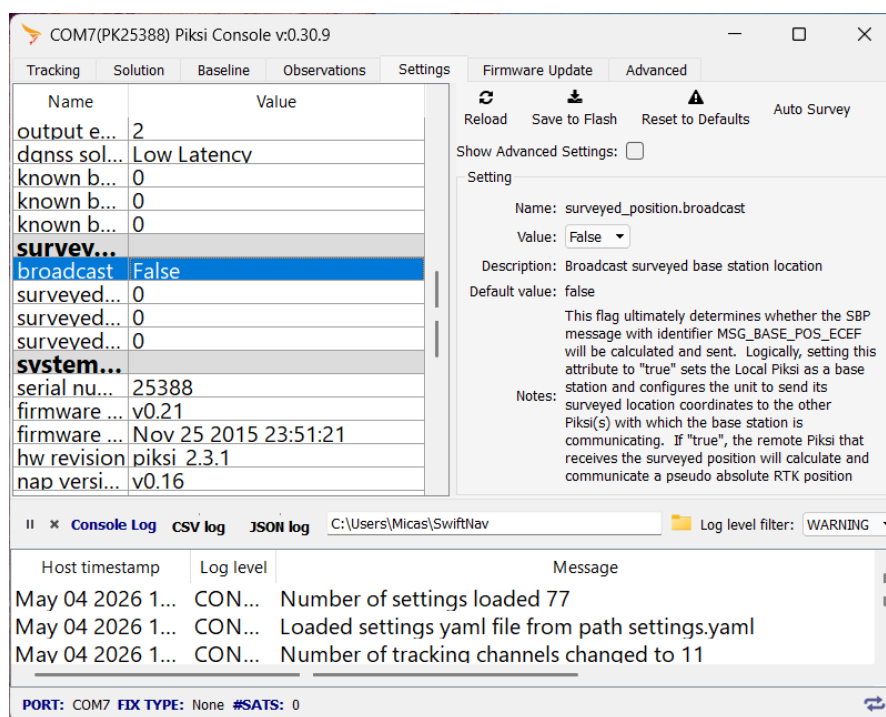


Figure 10: Program Setup for Rover

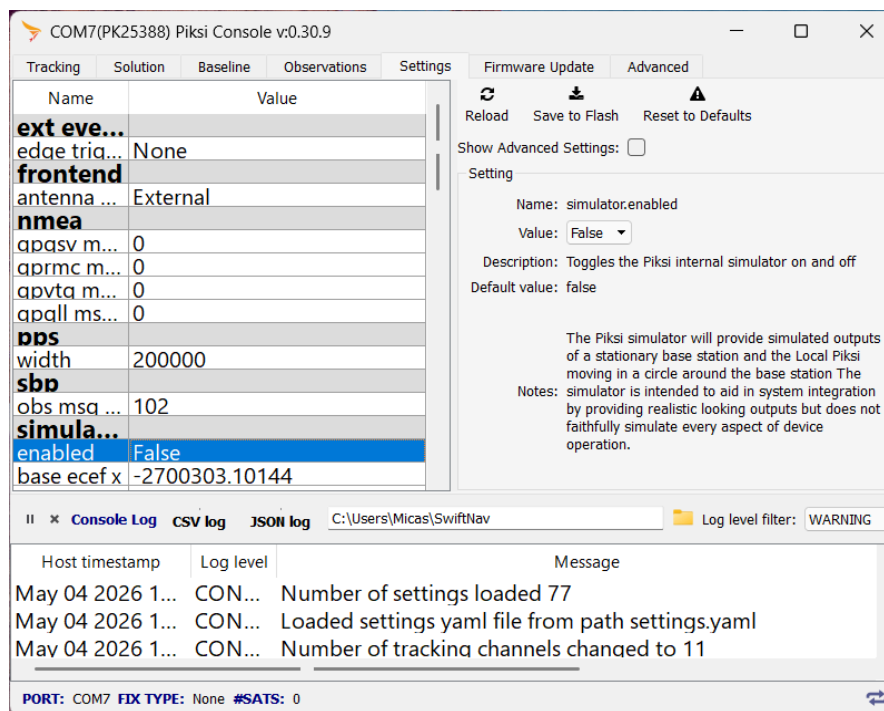


Figure 11: Program Setup to Disable extra NMEA Transmissions

RTK Software / Slave Setup

As of right now, the code for the slave board connection needs to be fixed, and/or redone. The Piksi is hardwired to send out data from UARTB at 10Hz, or once every 100 milliseconds, and take at most 80 milliseconds to completely send out its data packet. The slave code that was designed was meant to only take a complete sample once a second. It is designed to recognize the handshake, skip over the UTC Data, Store the Latitude and Longitude Data, and disable when the end transmission packet is sent.

An indicator was created that will light up when the 1 second timer goes off, and will disable when the data has been processed and stored. However, every time that the indicator goes off, it takes 8 seconds to disable. At the time of writing this, no solution has been found.

Troubleshooting procedure

1. System Architecture Overview

The Universal Remote Controller (URC), built around a PIC16LF1789 44-pin microcontroller, accepts joystick and button inputs and transmits control data to the Bloodhound vehicle over a wired UART link. On the vehicle, a Master Main Board built around a PIC16F1788 receives and decodes those UART packets, then forwards control data to a Slave Development Board, also a PIC16F1788, over a 100 kHz I2C bus. The Slave board generates PWM output signals that drive the DOCYKE power steering servo and two brushless speed controllers. Power is supplied by a 52V LiFePO4 battery pack. A DC/DC converter steps that voltage down to a regulated 24V rail for the servo and all onboard electronics.

2. SAFETY PRECAUTIONS

The 52V LiFePO4 battery can source hundreds of amps into a short circuit. Always disconnect the battery before probing or reworking any board. The following precautions must be observed at all times.

Engage the physical battery disconnect and BMS switch before any rework. Verify 0V between chassis ground and all power rails with a DMM before touching any PCB. Keep the vehicle on a stand with drive wheels clear of the ground during all motor tests. Do not operate the motors without securing all phase-wire connectors, as arcing can damage the speed controller. The 48V phase lines carry high frequency switching noise, so use only rated probes and differential inputs when scoping them. Wear safety glasses when probing live circuitry. Keep a Class C or CO2 fire extinguisher nearby during first power on after any wiring change.

3. TOOLS AND TEST EQUIPMENT

Digital Multimeter (DMM): Used for voltage, continuity, and resistance checks.

Oscilloscope, 2-channel, 100 MHz minimum: Used for PWM waveform capture and UART/I2C signal analysis.

PICkit 4 or MPLAB Snap: Used for programming and live debugging via ICSP.

MPLAB X IDE: Used for firmware build, breakpoints, and watch registers.

Bench Power Supply, 0 to 60V at 5A: Used as a safe stand in for the battery during low power tests.

Servo Tester or Signal Generator: Used to inject a known PWM signal to verify the DOCYKE servo independently of the firmware.

Caliper or ruler: Used to verify PCB component placement when reworking.

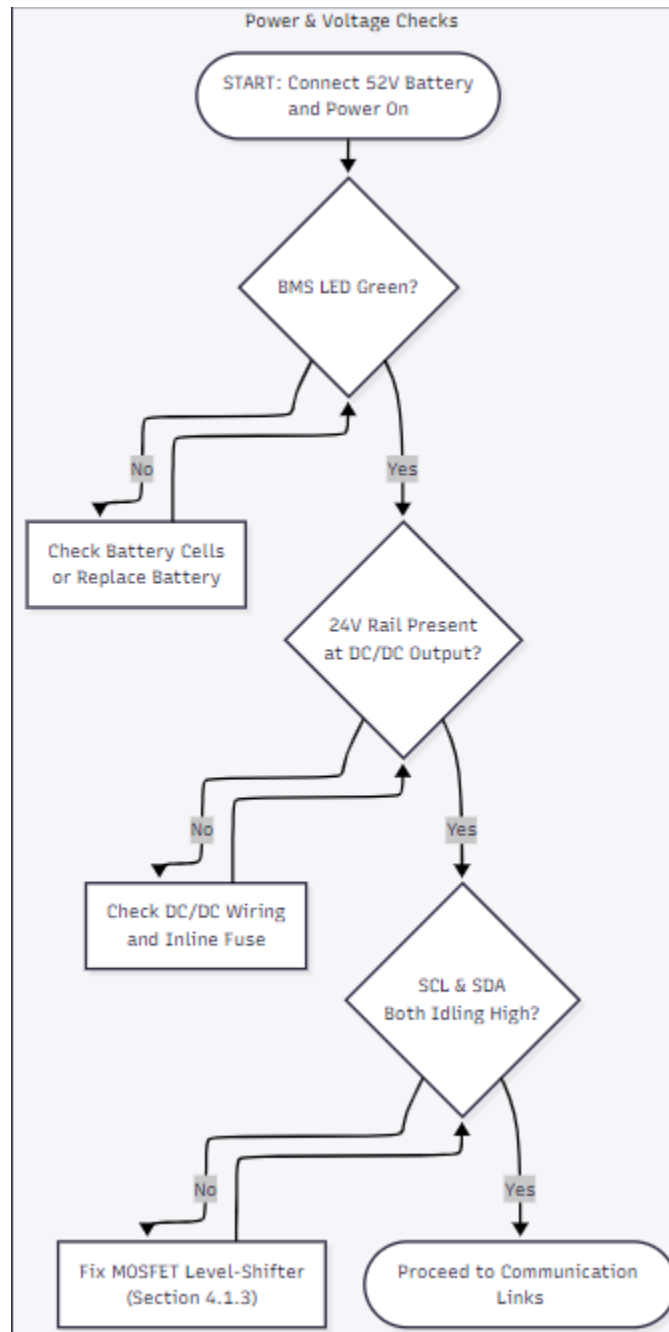
TROUBLESHOOTING FLOWCHARTS:

Figure 12: Troubleshooting Flowchart; Power and Voltage Checks

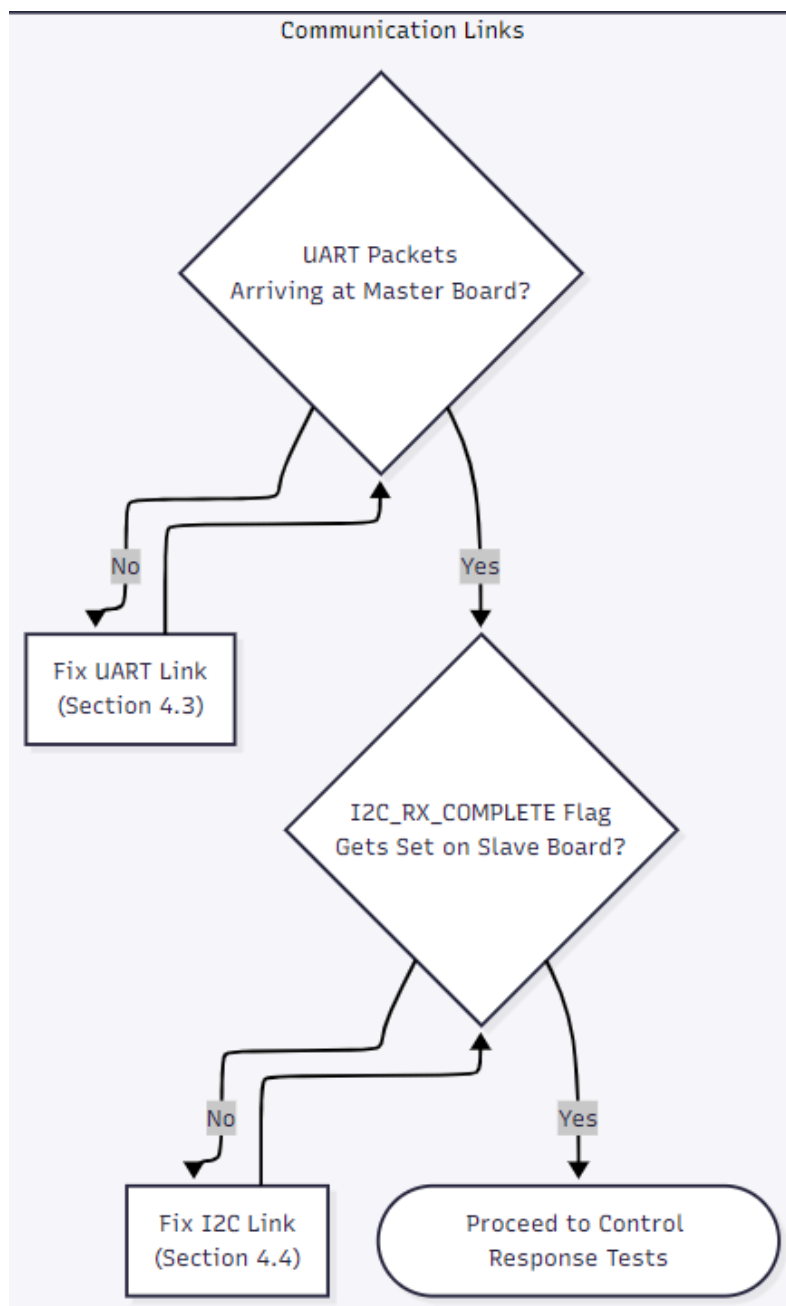


Figure 13: Troubleshooting Flowchart; Communication Links

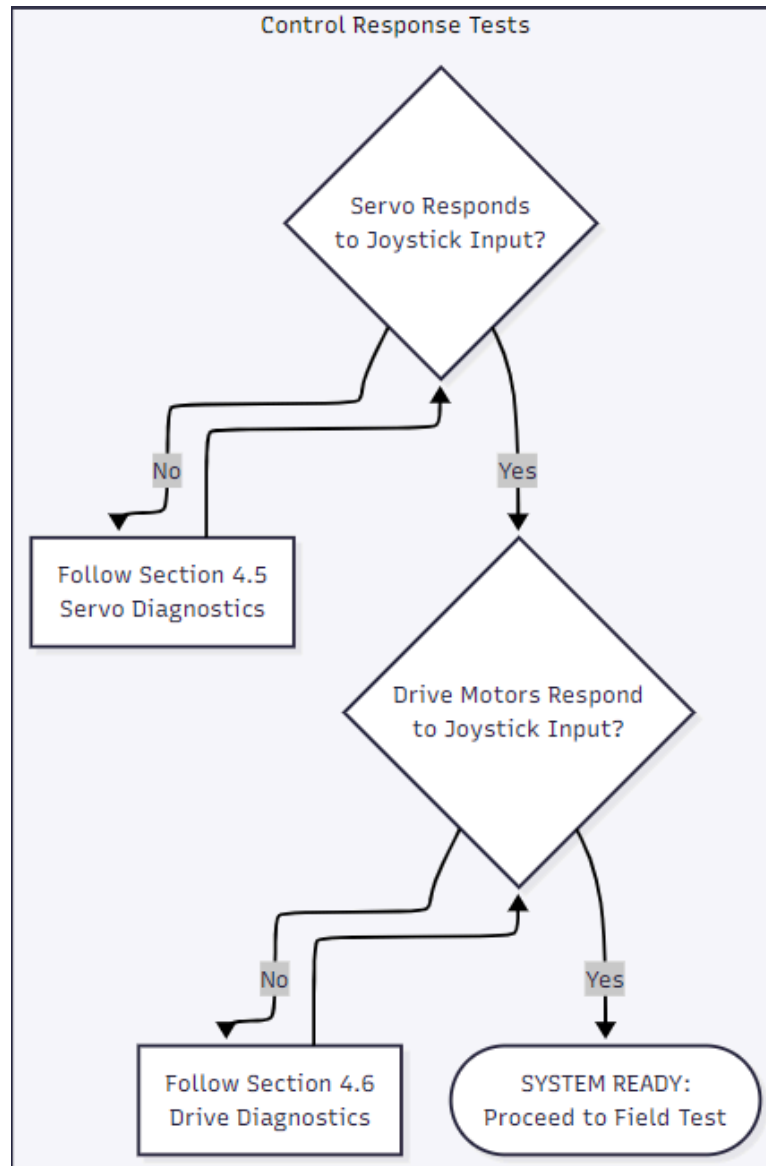


Figure 14: Troubleshooting Flowchart; Control Response Tests

4. TROUBLESHOOTING WALKTHROUGH

The following sections walk through the system layer by layer, starting at the power rails and moving up through microcontroller configuration, communication links, and actuator outputs. Each section identifies test points, expected voltages, normal waveforms, and known fault signatures.

4.1 POWER SYSTEM VERIFICATION

All voltage measurements are taken with the battery connected through the BMS unless otherwise stated. Nominal battery voltage is 52V. A fully charged LiFePO4 pack reads approximately 58.4V and a discharged pack reads approximately 44.8V. The DC/DC converter output target is 24V plus or minus 0.5V.

4.1.1 Battery and BMS

Test Point: BMS output terminals, B+ to chassis GND.

Step 1. Connect the battery and measure pack voltage at the BMS output terminals.

Expected result: 44 to 58V.

Step 2. Verify BMS indicator LEDs. Green indicates healthy. Red or off indicates a fault condition.

Step 3. Measure current draw at idle with no motors running. Expected result: less than 0.5A.

Step 4. Inspect all high current connectors for heat discoloration or corrosion.

Step 5. Verify battery cell balance voltage via the BMS display or cell tap wires.

Expected result: all cells within 50 mV of each other.

4.1.2 DC/DC Converter (30 to 60V input, 24V / 10A output)

Test Point: DC/DC OUT+ terminal to chassis GND.

Step 1. Measure DC/DC input voltage. This should equal battery pack voltage. Expected result: 44 to 58V.

Step 2. Measure DC/DC output voltage at the output terminals. Expected result: 23.5 to 24.5V.

Step 3. Apply a 5 ohm, 25 watt load resistor and re-measure output voltage. Expected result: output drops less than 0.2V under load.

Step 4. Check output ripple on the oscilloscope with AC coupling. Expected result: less than 200 mV peak to peak.

Step 5. Verify no heat sink temperature exceeds 70 degrees Celsius during a 5-minute run.

Normal waveform: With AC coupling on the oscilloscope, the DC/DC output should appear as a nearly flat line close to 0V with very slight high-frequency ripple. Any ripple exceeding 200 mV peak to peak indicates the converter is overloaded or has a failing output capacitor and should be investigated before powering any logic boards.

Bad waveform indicator: Ripple greater than 200 mV peak to peak, or an oscillating output voltage, suggests the converter is overloaded or the output filter capacitor has failed. Check load current on the 24V rail and verify there are no short circuits before proceeding.

4.1.3 I2C Level Shifter: Master Board Q1/Q2 MOSFETs

Known previous fault: Q1 and Q2, the N-channel MOSFETs in the Master board's I2C level-shifting circuit, were found to be unpopulated on the initial board. This caused both SCL and SDA lines to read 0V permanently, making I2C communication entirely non functional. These components must be verified before any I2C debugging is attempted.

Test Points: Gate of Q1 and Q2 (DMM probe), SCL line at the Master board I2C header, SDA line at the Master board I2C header.

Step 1. With the board powered, probe the gate of Q1 and Q2 with the DMM. Expected result: gate is driven to a logic level and is not floating.

Step 2. Measure SCL line voltage at the Master board I2C header with the bus idle. Expected result: 3.3V or 5V pulled high, not 0V.

Step 3. Measure SDA line voltage at the Master board I2C header with the bus idle. Expected result: 3.3V or 5V pulled high, not 0V.

Step 4. If 0V is found on SCL or SDA, inspect the Q1 and Q2 solder joints. If the components are missing, install them or bridge the drain-source pads with a bypass wire. Expected result: voltage present on both lines after the fix.

Bad condition indicator: SCL or SDA reading 0V at idle with the board powered and no active I2C transaction means the level-shifter MOSFETs are absent, incorrectly soldered, or shorted. The fix is to install the MOSFETs correctly or bridge the drain-source pads with wire to bypass the level-shifting stage.

4.2 MICROCONTROLLER VERIFICATION

4.2.1 Oscillator (16 MHz Internal)

All three boards; the URC, Master, and Slave, use the internal 16 MHz oscillator. Writing `OSCCON = 0x78` sets the IRCF bits to 1111, selecting 16 MHz. No external crystal is fitted on any board.

Step 1. Attach the PICKit 4 to the target board's ICSP header in MPLAB X and attempt to read the device ID. Expected result: device ID detected, board is not blank.

Step 2. Read the OSCCON register in live debug mode. Confirm IRCF bits 6 through 3 are all 1. Expected result: OSCCON reads 0x7x.

Step 3. If CLKOUTEN is enabled in CONFIG1 for debug purposes, probe the CLKOUT pin with the oscilloscope. Expected result: a clean 4 MHz square wave, which is F_{osc} divided by 4.

Step 4. Flash a simple LED blink sketch with a known 1-second period and verify timing with a stopwatch. Expected result: LED toggles exactly every 1 second, confirming 16 MHz operation.

Normal waveform: A clean 4 MHz square wave with approximately 50% duty cycle on the CLKOUT pin confirms the oscillator is running correctly and the PIC is executing code.

Bad waveform indicator: No signal on CLKOUT, or a frequency other than 4 MHz, indicates an oscillator configuration problem. Verify OSCCON is being written correctly in the INITIALIZE routine and that CONFIG1 selects the INTOSC option (`_FOSC_INTOSC`).

4.2.2 Timer 2 Configuration — Slave Board

Known previous fault: T2CON was left at 0x00, which produced timer interrupts approximately every 62 microseconds instead of the intended 4 milliseconds. This caused the I2C communication timeout counter to reach its threshold almost immediately after boot, before any valid I2C data could arrive, placing the system in a permanent timeout state on every startup. The correct configuration is T2CON = 0x79, which sets the prescaler to 1:4 and postscaler to 1:16, with PR2 = 249. This produces the intended 4 millisecond interrupt period.

Test Point: Slave board pin RA0, toggled inside the TMR2 ISR as a debug output.

Step 1. Read the T2CON register in the debugger on the Slave board. Expected result: T2CON = 0x79.

Step 2. Read the PR2 register. Expected result: PR2 = 249.

Step 3. Toggle RA0 inside the TMR2 interrupt service routine and probe with the oscilloscope. Expected result: approximately 4 millisecond period square wave.

Step 4. If the interrupt period measures approximately 62 microseconds, T2CON was 0x00. Correct the prescaler and postscaler and verify the period becomes 4 milliseconds.

Step 5. Verify the I2C_TIMEOUT_COUNT does not reach its threshold within 1 second of power-on with no UART connected. Expected result: the count increments slowly and reaches timeout in approximately 0.5 seconds.

Normal waveform: A square wave at approximately 250 Hz, with each half-period close to 2 milliseconds, observed at RA0 when toggled once per TMR2 interrupt.

Bad waveform: fast interrupt; A square wave at approximately 16 kHz, corresponding to a period of roughly 62 microseconds. This is the failure mode when T2CON is not properly initialized. At this rate any counter driven by the TMR2 interrupt overflows far faster than intended, causing the I2C watchdog timeout to fire immediately on every boot.

4.2.3 Timer 2 Configuration — Master Board

The Master board's Timer 2 drives two counters: CONNECT_CHK_COUNT, which detects loss of UART communication, and I2C_COM_DELAY, which paces I2C transmissions to occur every 96 milliseconds. The Master uses PR2 = 250 and T2CON = 0x00, producing an interrupt period of approximately 32 milliseconds as verified empirically with UART traffic.

Step 1. Read PR2 and T2CON on the Master board in the debugger. Expected result: PR2 = 250, T2CON = 0x00.

Step 2. Confirm I2C_COM_DELAY reaches a threshold of 3, causing the I2C send flag to be set every approximately 96 milliseconds. Expected result: NEW_RX_DATA bit 1 toggles approximately every 96 milliseconds.

Step 3. Confirm CONNECT_CHK_COUNT resets to 0 when a valid UART packet arrives. Expected result: count clears on each good packet received.

4.3 UART COMMUNICATION — URC TO MASTER

Baud rate: 115,200 bps, 8-N-1 framing. Register settings: SPBRG = 34, SPBRGH = 0, BRG16 = 1, BRGH = 1. Physical interface: wired UART lines between URC TX on RC6 and Master RX on RC7.

Expected UART waveform: The TX line idles high. Each byte begins with a start bit that pulls the line low, followed by 8 data bits transmitted LSB first, followed by a stop bit that returns the line high. At 115,200 bps, each bit occupies 8.68 microseconds. A single byte therefore takes approximately 86.8 microseconds to transmit. A complete six-byte packet occupies approximately 521 microseconds on the wire.

Normal waveform: The idle line sits at logic high, approximately 3.3V. On each byte transmission the line drops to 0V for the start bit, transitions through the 8 data bit levels at 8.68 microsecond intervals, then returns high for the stop bit. On the scope this appears as a burst of transitions for each byte with the line returning to idle high between packets.

Bad waveform: inverted signal; If the line idles low, the UART polarity is inverted. Check the SCKP bit in BAUDCON, which should be 0 for non-inverted, and verify the physical connection between URC TX and Master RX is not reversed.

Bad waveform: wrong bit period; If the bit period is not 8.68 microseconds, the baud rate does not match between boards. Verify SPBRG = 34 on both the URC and Master boards and confirm BRG16 and BRGH are both set to 1. A FERR framing error bit set in the RCSTA register confirms a baud rate mismatch at the receiver.

Step 1. Probe the URC TX pin with the oscilloscope. Confirm idle state is high. Expected result: line idles at logic high.

Step 2. Transmit one packet and verify the bit period is 8.68 microseconds. Expected result: bit period matches 1 divided by 115200.

Step 3. Probe the Master RX pin and verify the signal arrives with correct level and polarity. Expected result: signal matches the URC TX.

Step 4. In the debugger, confirm RCSTA reads: SPEN = 1, CREN = 1, FERR = 0, OERR = 0. Expected result: RCSTA = 0x90 with no error flags set.

Step 5. Monitor BYTE_COUNT in the debugger while packets are being received. Expected result: BYTE_COUNT increments from 0 through 6 and then clears for each packet.

Step 6. Confirm the handshake byte 0x24 is detected correctly at BYTE_COUNT = 0. Expected result: the HANDSHAKE_VERIFIED branch is taken.

Step 7. Confirm NEW_RX_DATA bit 0 sets after the 6th byte is received. Expected result: flag asserts correctly.

Step 8. If the OERR overrun error bit is set, clear it by toggling CREN: clear bit 4 of RCSTA then set it again. Expected result: error clears and reception resumes.

Step 9. If the FERR framing error bit is set, a baud rate mismatch is indicated. Verify SPBRG = 34 on both boards. Expected result: FERR = 0 after the baud rate is corrected.

4.4 I2C COMMUNICATION — MASTER TO SLAVE

I2C speed: 100 kHz. SSPADD on the Master = 0x27. Slave write address: 0x20. Slave read address: 0x21. Both boards have their DIP switches set to address 0x00.

Test Points: SCL at the Master I2C header pin 1, SDA at the Master I2C header pin 2.

Expected I2C waveform: On the oscilloscope, SCL appears as a 100 kHz square wave with a 10 microsecond bit period and approximately 50% duty cycle during data transfer. SDA transitions only while SCL is low. A START condition is a high-to-low transition on SDA while SCL is high. A STOP condition is a low-to-high transition on SDA while SCL is high. The

address byte transmitted to the Slave is 0x20, which in binary is 0010 0000. It is sent MSB first, followed by the write bit (0), and then the Slave acknowledges by pulling SDA low on the 9th clock cycle.

Normal waveform: SCL pulses at 100 kHz throughout the entire transaction. SDA shows the address bits, then the data bits of all four data bytes, with the Slave pulling SDA low as an ACK on the 9th clock after each byte. SDA returns to idle high after the STOP condition.

Bad waveform: no clock; SCL stuck low or not toggling means the Master's MSSP peripheral is not generating a clock. Check that I2C master mode is enabled (SSPCON = 0x28) and that SSPADD = 0x27. Also verify that the Q1 and Q2 level-shifter MOSFETs are installed as described in Section 4.1.3.

Bad waveform: no ACK; If SDA remains high on the 9th clock pulse after the address byte is sent, the Slave is not acknowledging. Verify the Slave board is powered, that its SSPADD register matches the address being sent by the Master (0x20), and that the Slave's SSP peripheral is enabled (SSPCON bit 5 = 1).

Step 1. Verify the SCL pull-up resistor is present to the 3.3V or 5V rail, typically 4.7 kilohms. Expected result: pull-up present and bus idles high.

Step 2. Verify the SDA pull-up resistor is present. Expected result: pull-up present and bus idles high.

Step 3. Probe SCL and confirm the level-shifter MOSFETs Q1 and Q2 are installed and functioning as described in Section 4.1.3. Expected result: SCL not stuck at 0V.

Step 4. Probe SCL with the oscilloscope during an I2C write attempt and verify the 100 kHz clock is present. Expected result: 100 kHz clock visible.

Step 5. Probe SDA and verify the address byte 0x20 is transmitted correctly, bit pattern 0010 0000. Expected result: address byte confirmed on the logic analyzer.

Step 6. Verify the Slave board SSPADD register reads 0x20 in the debugger. Expected result: SSPADD = 0x20.

Step 7. Confirm the Slave SSP1IF flag, PIR1 bit 3, sets on each received byte. Expected result: SSP1IF asserts once per byte.

Step 8. Confirm the D_NOT_A bit, SSPSTAT bit 5, transitions correctly: it should read 0 for the address byte and 1 for each subsequent data byte. Expected result: D_NOT_A shows the correct sequence.

Step 9. Check the SSPCON1 CKP bit and confirm the clock is released, meaning BSF SSPCON1 bit 4, after each byte is processed by the ISR. Expected result: CKP is re-enabled after each byte.

Step 10. After 4 data bytes are received, confirm I2C_RX_COMPLETE bit 0 reads 1 in the Slave debugger. Expected result: flag asserts after the 4th byte.

Step 11. If an ACK is not received, meaning SSPCON2 bit 6 reads 1, check that the Slave address matches and that the Slave board is powered. Expected result: ACKSTAT clears after the issue is resolved.

4.5 DOCYKE POWER STEERING SERVO

The DOCYKE servo is connected to Slave board pin RA4. The PWM signal has a 20 millisecond period, corresponding to 50 Hz. Pulse width determines steering position: 0.5 ms is

full left deflection, 1.5 ms is center, and 2.5 ms is full right deflection. The servo is supplied by 24V from the DC/DC converter and accepts a signal voltage of 3.3 to 5V.

Test Point: Slave board pin RA4 measured against chassis ground.

Normal waveform at center position: The signal is high for 1.5 milliseconds, then low for 18.5 milliseconds, repeating every 20 milliseconds. On the scope this appears as a narrow positive pulse at a 50 Hz rate.

Normal waveform at full left, 0.5 ms pulse: The high portion narrows to 0.5 milliseconds with 19.5 milliseconds low. This corresponds to JOY2_LR at or below the lower threshold of 0x28.

Normal waveform at full right, 2.5 ms pulse: The high portion widens to 2.5 milliseconds with 17.5 milliseconds low. This corresponds to JOY2_LR at or above the upper threshold of 0xC8.

Bad waveform: pulse stuck at 1.5 ms regardless of joystick input: I2C data is not being received or the received data is not being decoded. Check the I2C link per Section 4.4 and verify I2C_RX_COMPLETE is being set and cleared correctly in the main loop.

Bad waveform: RA4 stuck low with no pulse: TRISA bit 4 may be set to input instead of output. Verify TRISA = 0x00 in the debugger. Also confirm Timer 2 is running by checking T2CON bit 2 is set.

Bad waveform: incorrect period, not 20 ms: Timer 2 configuration is incorrect. Verify T2CON = 0x79 and PR2 = 249 on the Slave board, and confirm T20MS_COUNT_1 and T20MS_COUNT_2 are both initialized to 200 in the INITIALIZE routine.

Step 1. Inject a known 1.5 ms PWM signal with the servo tester and verify the servo moves to center. Expected result: servo reaches mechanical center.

Step 2. Inject a 0.5 ms pulse and verify the servo moves to the full left limit. Expected result: full left deflection.

Step 3. Inject a 2.5 ms pulse and verify the servo moves to the full right limit. Expected result: full right deflection.

Step 4. Remove the servo tester, probe Slave pin RA4 with the oscilloscope while the system is running with JOY2_LR at 128 (0x80). Expected result: 1.5 ms pulse with a 20 ms period.

Step 5. Push joystick 2 to full left and verify the pulse narrows toward 0.5 ms. Expected result: pulse width less than 1.0 ms.

Step 6. Push joystick 2 to full right and verify the pulse widens toward 2.5 ms. Expected result: pulse width greater than 2.0 ms.

Step 7. If RA4 is stuck low, confirm TRISA bit 2 = 0 (output mode) in the debugger. Expected result: TRISA = 0x00.

Step 8. If the pulse period is incorrect, verify the T20MS_COUNT thresholds and Timer 2 configuration per Section 4.2.2. Expected result: period corrects to 20 ms after the timer fix.

Step 9. Check the 24V supply voltage at the servo power connector. Expected result: 23.5 to 24.5V.

Step 10. Verify all three servo wires; PWM signal, power, and ground, are fully seated in the connector. Expected result: all three wires present and secure.

4.6 DRIVE MOTORS AND SPEED CONTROLLERS

Two brushless 48V hub motors are each controlled by a separate 2-mode brushless speed controller. Battery voltage rail: 52V LiFePO4. Speed control input to each controller is a variable PWM or 0 to 5V analog throttle signal generated by the Slave board. Always verify the wheels are off the ground before testing motor rotation.

Speed Controller Wire Assignments:

Red and black power wires connect to battery B+ and B-. Verify polarity before connecting. The three phase wires connect to the hub motor phase terminals. All three must be connected. The electric door lock wire must always be connected to B+ as it is required for the controller to enable. The speed control handle wire carries the throttle PWM from the Slave board, a 0 to 5V or PWM signal. The high level brake wire is a digital output from the Slave, active high to engage braking. The reverse gear wire is a digital output from the Slave, pulled low to engage reverse. The 5V signal wires in blue, green, and yellow carry hall sensor signals from the motor back to the controller for phase angle feedback.

Expected phase waveform while motor is running: Between any two phase wires on the oscilloscope, a trapezoidal or sinusoidal waveform should be visible at approximately 48V peak. The frequency of this waveform will vary with motor speed. This is the BLDC commutation waveform produced by the controller.

Bad waveform: flat DC on phase wires with motor not spinning: The controller is not enabled. Verify the electric door lock wire is connected to B+. Also verify the throttle signal is above the controller's minimum enable threshold.

Step 1. Verify battery voltage at the controller B+ input. Expected result: 44 to 58V.

Step 2. Verify the electric door lock wire is connected to B+ and check continuity.

Expected result: continuity to B+ confirmed.

Step 3. Apply a known 1.5V throttle signal to verify the stop state. Expected result: motor is stationary.

Step 4. Increase throttle to approximately 2.5V and verify the motor spins forward. Expected result: motor spins smoothly with no grinding.

Step 5. Pull the reverse wire low, apply throttle, and verify reverse rotation. Expected result: reverse confirmed.

Step 6. Measure phase voltage between any two phase wires with the oscilloscope during a run. Expected result: trapezoidal or sinusoidal waveform at approximately 48V peak.

Step 7. Verify the hall sensor 5V signals on the blue, green, and yellow wires toggle as the motor rotates using a logic probe. Expected result: all three signals toggle.

Step 8. If the motor does not respond, check the learning wire by connecting it to ground to reset the controller. Expected result: motor responds after reset.

Step 9. If the controller overheats, check the current limit setting, which should be in the 26 to 32 amp range. Expected result: temperature remains below 70 degrees Celsius at rated load.

5. END-TO-END SYSTEM TEST

This section should only be performed after all individual subsystem checks in Sections 4.1 through 4.6 pass. The vehicle must be on a stand with wheels clear of the ground throughout this test.

End-to-End Test Steps

Step 1. Power on the URC and confirm the controller LEDs are active. Expected result: URC powered and operational.

Step 2. Power on the Bloodhound and verify the 24V rail is present, the BMS LED is green, and no fault LEDs are illuminated. Expected result: all power rails nominal.

Step 3. Set all joysticks to center and wait 3 seconds. Expected result: servo holds center position with no drift.

Step 4. Move Joystick 2 slowly to the left and verify the steering moves left. Expected result: left steering confirmed.

Step 5. Move Joystick 2 slowly to the right and verify the steering moves right. Expected result: right steering confirmed.

Step 6. Move Joystick 1 slowly upward and verify both drive motors spin forward. Expected result: forward motion confirmed.

Step 7. Move Joystick 1 downward and verify reverse rotation on both motors. Expected result: reverse motion confirmed.

Step 8. Release all joysticks and verify the motors coast to a stop within 2 seconds. Expected result: motors stop.

Step 9. Simulate a UART communication loss by unplugging the UART cable and verify the Slave's I2C_COM_TIMEOUT fires within approximately 1 second, stopping the motors. Expected result: motors stop within 1 second of link loss.

[Step 10](#). Reconnect the UART cable and verify the system recovers normal operation without requiring a reboot. Expected result: normal operation resumes automatically.

6. KNOWN ISSUES AND RESOLUTIONS LOG

Issue 1. Symptom: SCL and SDA stuck at 0V. Root Cause: Q1 and Q2 MOSFETs were not populated on the Master board's I2C level-shifting circuit. Resolution: Install the MOSFETs or install a bypass wire across the drain-source pads. Status: Resolved.

Issue 2. Symptom: CONNECT_TIMEOUT fires immediately at every boot. Root Cause: T2CON = 0x00 produced approximately 62 microsecond interrupts, causing the timeout threshold to be reached before any UART data arrived. Resolution: Set T2CON = 0x79 and PR2 = 249 to produce the correct 4 millisecond interrupt period. Status: Resolved.

Issue 3. Symptom: All Master UART and I2C firmware replaced between development sessions. Root Cause: An external AI tool (Grok) overwrote the verified working firmware with an unrelated heartbeat test. Resolution: Restore from version history. Do not allow external AI tools to modify this codebase. Status: Resolved.

Issue 4. Symptom: Servo twitches randomly at startup. Root Cause: The PWM output begins before any I2C data arrives, and the count registers are uninitialized at that point. Resolution: Initialize STOP_COUNT to the 1.5 ms value inside the INITIALIZE routine before enabling interrupts. Status: Open, verify after next build.

Issue 5. Symptom: Motor controller does not enable. Root Cause: The electric door lock wire was not connected to B+. Resolution: Connect the door lock wire to the battery positive rail. Status: Open, verify wiring.

7. FUTURE ADDITIONS

The following subsystems have been identified for future integration. Each will require its own test procedure section with test points, expected voltages, and waveforms to be added to this document once the hardware is installed and tested.

IMU and Accelerometer: Perform an I2C address scan to confirm the device responds on the bus. Verify the sample rate and data output register format. Check axis orientation against the vehicle frame for correct interpretation of pitch, roll, and yaw data.

Chemical Sensors for CO and CO2: Calibrate the analog output against a known gas concentration reference. Verify ADC readings on the Slave board correspond correctly to sensor output voltage at known concentration levels.

Bidirectional Telemetry: Add a downlink UART or wireless module. Verify battery voltage and motor current telemetry packets are received and displayed correctly at the operator station.

Conclusion / Recommendations

The Bloodhound RCEV project achieved its primary objectives and can be considered a successful capstone. Core functionality was successfully implemented, including reliable wireless UART-to-I2C communication, joystick-controlled forward/reverse driving and left/right steering, and essential safety features such as I2C timeout with automatic brake engagement.

The main roadblock encountered was suboptimal time management. Many work sessions were spent determining the next steps rather than executing planned tasks. A more detailed day-by-day plan with specific, actionable items would have enabled smoother workflow transitions and stronger documentation for future use. While daily activities were documented, greater detail on specific achievements and complications would have enhanced the engineering log.

Several modular expansions remain. The circuit panel is fully prepared for headlight and rear light integration, which requires only mounting the lights and implementing button controlled firmware logic. Following this, GPS and LiDAR systems can be added, building on foundational work by students of the Spring 2026 semester.

Opportunities for improvement include a complete redesign of the steering system. The DOCYKE servo, while functional, proved suboptimal for this application and was the source of multiple issues, including mechanical stress on the plastic coupling. Replacing it with dual independent servos mounted directly to the tie rods would provide more responsive, precise steering and eliminate unnecessary mechanical linkages.

This project offers strong potential for future RCET work. The modular three board architecture (URC, Master, and Slave), robust communication framework, and power distribution design can serve as a reusable platform for other unmanned ground vehicle projects. The documentation and troubleshooting procedures developed here will also benefit subsequent teams working on similar embedded control or remote operated systems.

Overall, the Bloodhound project effectively integrated soldering, programming, wiring, mechanical design, problem solving, and documentation skills developed throughout the RCET program. It provided valuable new experiences, such as vehicle alignment, and significantly strengthened confidence in tackling complex engineering challenges.

Cost analysis

Team Member	Hours/ day	Total Days	Total Hours	Cost/ hr	Total
Noah Holloway	~5.5	31	170.5	\$50	\$8,525
Micah Spencer	~4	10	40.5	\$50	\$2,025
Total	9.5	41	211		\$10,550
Total + BOM					\$14,026.50

(see Appendix for BOM and Engineering Log for itemized detail)

Over the eight week project, the majority of time was spent on hardware assembly (soldering boards, servo mounting, power wiring, and chassis integration), firmware development for UART/I2C communication, joystick to PWM control, and extensive troubleshooting of timing issues, communication failures, solder joints, and component placement. Later weeks focused on system integration, motor/servo tuning, brakes, and final clean wiring to achieve basic drivable functionality.

Significant time was lost to recurring issues like incorrect timer configurations, address mismatches, and premature full system testing. Efficiency could have been improved by verifying all board components and power rails early, conducting more isolated bench testing before vehicle integration, implementing stricter version control and documentation from the

start, and allocating dedicated time upfront for scope/protocol training and systematic debugging processes. This would have reduced repeated debug cycles and allowed more focus on advanced features like headlights, which unfortunately was unable to be integrated in the time allotted for this project.

Engineering Log

Date	Accomplished	Complications	Goals
3-13-26	- Found 50% on DOCYKE servo motor - Aligned & pinned servo to steering column - Achieved manual steering with 'CCPM servo consistency measter' servo tester - Aligned front wheels (initially 40" front and 38.5" back, aligned to 39.5" front and 39.5" back)	- Cracked DOCYKE servo PCB and forced motor & servo modes into high & low speed motor mode - Broke drill bit - Didn't put a set screw into steering column servo	- Become familiar with universal controller: - Figure out joystick read/write data packets - Program Joystick 1 as forward and reverse - Program either joystick two or three as left and right steering - Solder universal remote main board (x1) - Solder rs232 adapter board(x2)

3-16-26	• Soldered Master receiver/ main board • Found joystick hex value ranges on universal controller • Collaborated with Rudy on which buttons/joysticks will be used for cameras	• Bent PIC16f1788 legs while soldering had to throw away and replace it	• Solder both rs232 adapter boards • Figure out controller communication protocols for wireless data transmission • Figure out/write program to assign joystick data to throttle and steering data
3-17-26	• Soldered both rs232 adapter boards • Installed MPLABS V.5.2 to read universal remote code	• Having file/opening project issues within MPLAB, 6.2 & 5.2 there are several build errors. Cannot find files.	• Fix file issues with MPLAB 5.2 • Program uni remote & main board to observe i2c joystick data
3-18-26	• Programmed universal remote	• Built an rs232 communication	• Communicate universal

	and main board, fixed file issues • Observed joystick L&R bumper and button data on oscilloscope • See Appendix Figure () for joystick data findings	circuit to interface with uni remote companion test software using the MAX3232 ic. The IC got hot for seemingly no reason. Moved circuit config to different parts of the breadboard and it worked. Middle of breadboard(25-35) might be broken	remote and Main board with rs232 cable. Attached and get Bloodhound motors moving • After verifying that if communication disconnects the motors stop then proceed to using wireless communication • Solder PIC dev board from Shane • Main board will connect to PIC dev board, servo and motor outputs taken from PIC dev board, not main board
--	--	--	--

			Start with slave code for dev board, but it will probably need tweaking
3-19-26	- Soldered pick dev board - Program general slave code on devboard (all three boards now programmed)	- Master main board was current limiting C2 was soldered on backwards - Used a thermal gun to find c2's issues. Re-soldered new C2 correctly	- Tweak Master main and slave devboard code to allow for joystick 2. On the uni remote to control left and right steering - Program and connect boards according to specifications
3-20-26	Programmed all three boards and wired them to control the DOCYKE servo with joystick 2	- The uni remote did not move the servo when connected - The problem was isolated to master main code. It's most	- Reprogram Master main board with correct timing. Config info - Get servo to

		likely an oscillator/timing/prescaler, post scaler config error	move with joystick 2 Get motor to move forward and backward with joystick 1
3-30-26	NA	Tried various different code iterations but none of them worked	Working on troubleshooting, RA0 has a sawtooth wave with a period of 1.5 ms and a VPP of 0.7VPP. Run with that and figure it out/ isolate where the issue is coming from
3-31-26	Programmed D4-D10 leds on Master main board to act as debug lights slowly but surely figuring out	Don't seem to be measuring any i2c data, which seems to mean that there's an issue with the Master main code,	Ensure scope settings are correctly configured to read i2c to verify the issue is with

	which parts of the code are executing and which are not	but it's entirely possible that I just don't have the right scope settings configured to read i2c	the board Slowly and systematically fix the master main code to convert the rs232 data into i2c data
4-1-26	- Got i2c data after resoldering r3 on Master main board - Got joystick 2 to control the servo for about 30 seconds. Then it stopped working for some reason	- Unsatisfactory solder joint on r3 caused i2c to not be sent - Might have pulled too much current for the servo and potentially damaged the power supply	Connect servo to 52 volt battery and try joystick 2 again
4-2-26	- Fixed the slave code servo period (20.8ms to 19.8ms) - Consistently turned the steering	Stripped out the plastic 3D printed coupler D shaft. So the steering column is not driven by the servo anymore	Reconnect new steering column coupler to servo and remount it to the Bloodhound

	with joystick 2		• Test and verify that joystick 2 reliably turns the steering while the Bloodhound is on the ground • Program new Max and min PW limits for the servo adjust as needed for full range without hyperextending the tie rods
4-3-26	• Mounted the new coupler to the servo • The torque is more than enough to turn the wheels on the ground	• Startup addresses aren't the same for some reason • Servo wasn't receiving pwm at startup because of different addresses and went into free run mode and tried to spin like a motor	• Mount all relays and power supplies to the Bloodhound for non-tethered testing • Right power-up safety code to only give power to the servo if

		and began smoking	pwm is received • Write code to fix and verify the addresses
4-6-26	• Successfully set up git repo and branches for drive, thermal camera, GPS, and lidar • Verified that the servo is not broken. Connected servo tester • Completed wiring diagram for 52v battery connection	• Need to change code to control joystick 1 for forward and reverse and left and right steering	• Connect battery bus bars and step down converters and hookup and verify functionality of universal remote, Master, and slave boards with setup • Change master or slave code to use joystick one instead of joystick 2 • Rear headlights should come on automatically

			when joystick one one is in reverse - Joystick to button press should activate front headlights - Right power up safety code for servo pwm and fix address bug
4-7-26	Connected most of the mounting hardware and buses to battery and temporarily mounted things to the Bloodhound	NA	- Finish mounting battery hardware and terminals and test servo code for functionality - Same goals from 4-6-26
4-8-26	Finished mounting hardware and connecting cables for complete battery power	Servo doesn't completely return to center when releasing joystick 2. I think that is a	Adjust left and right steering limits to verify that steering is not a code issue

	- Added fuses to 24v bus to allow it to work - Fine-tuned steering servo code for the most part to allow for joystick 2 to turn the steering servo and return back to center	hardware issue, not a servo code issue - Servo began smoking again after getting jammed all the way to the right when the signal wire was abruptly removed while powered on. This is the same issue as before - Steering right. Doesn't steer as much as it does to the left	- Program joystick one to control, forward and reverse on motors - Potentially develop code to only use one half motor power to start with to make sure it won't be too fast - Program brakes and give it a dead zone
4-13-26	- Recovered old working code - Note that MPLAB stores code history if you ever need to find old working code and have forgotten to push	- Joystick 2. Currently isn't working to move the steering servo - Could be that I cannot use time or two for both the motor and servo	

	to git		
4-14-26	Worked with Peyton to create better subroutine for time or two with lookup table	- Learned not to program slave board with servo signal wire plugged in - Blew up PORTA bits 2-3	Change literal value in slave code to joystick data that is converted from lookup table and have that loaded into servo or motor count/ period
4-15-26	Servo pwm is happy and works well	Issue with the math for dead zone thresholds for the motor	
4-16-26	Worked with Peyton to fix and fine-tune motor and steering thresholds for gradual forward and reverse & L+R steering		Need to fine-tune or create subroutine to allow for both reverse direction and braking when

	• Drove around the lab slowly • Got brakes hooked up and working • Verified i2c timeout engages brakes		pulling back on the joystick
4-17-26	• Began the process of rewiring. Everything on the Bloodhound to make it look nicer and more clean • Made chassis ground		• Shorten wires • Before rewiring, anything work with Shane to figure out panel size and shape for mounting hardware
4-27-26	• Wired power and key control from speed controllers to battery • Secured positive distribution block to frame		• Once side panel is complete, begin mounting boards and locks to panel • Once mounted, begin wiring • Once wired, power remote

			with 5v battery and replace wired connection with wireless connection
4-29-26	- Made cables and wired mostly everything for normal drive operation - Updated slave code to replicate correct forward reverse and break code for both motors		- Need to wire motor forward motor reverse and brake for both motors/speed controllers - Need to wire brake relay - Need to wire headlight, relays and mount headlights and right headlight code
4-30-26	Finished wiring/mounting everything but	Left motor wasn't working at all with either speed	Reprogram or verify current programming for

	there probably won't be time to implement headlights	controller Reverse speed is too slow to reverse while on the ground	left motor on port B
5-1-26	- Fixed programming on port so now both motors work correctly - Figured out left motor slow speed issue by connecting speed controller learning wires together		Finish report and presentation and if there's time then implement headlight mounting and code.

Date	Accomplished	Complications	Goals
4-16-26	- Did preliminary research on given Piksi module - Located original documentation on Piksi		- Become familiar with RTK systems - Determine needs for the Piksi

4-17-26	• Did further research on how to set up the RTK function • Determined that a base station was required to operate RTK • Figured out how the Piksi logs it's files in the program	• Discovered that there wasn't a readily available base station to piggyback off of • Discovered that a subscription was needed for private networks	• Figure out how to set up the RTK • Determine if there is a readily available RTK network • Figure out how the Piksi saves data
4-22-26	• Downloaded correct console to program Piksi • Started learning how to operate the console	• The recommended Swift Console couldn't run the given Piksi • Weather was overcast, so testing wasn't possible	• Download the necessary console to program Piksi • Start learning how to use console
4-23-26	• Updated console that fixed bugs • Figured out how the data was sent across the UART • Set up the Piksi to	• Console wasn't up-to-date • Preset data transmission used proprietary code • Preset Baud rate	• Update Piksi Console • Figure out how to process GPS data • Establish correct

	send out data in NMEA Adjusted Baud Rate so NMEA data could be read	for NMEA causes data errors	GPS data output
4-24-26	- Set up Base station - Field tested the base station and rover - Started determining how to set up the slave	- Data displayed on the console gets removed after a set amount of time - Had to field test solo, so errors could have occurred unknowingly	- Field test base station - Determine if rover data can be read at base station - Determine how the slave board should be connected
4-27-26	- Verified requirements with project lead - Main loop, TMR1 logic and Handshake logic were started	Had to locate manual for PIC16F1788	- Start working on the slave program - Determine how the data was going to be processed
4-28-26	Verified power		Verify UART

	connection through UART • Set up overrun and framing error logic • Set up subroutines to store necessary data		power and data connections • Finish setting up slave code
4-29-26	• Fixed instruction timing issue	• Built in indicator showed data taking 6 seconds to store • Data instructions were operating in 16MHz, instead of 32MHz	• Troubleshoot slave code • Fix issues with the instruction timing
4-30-26	• Fixed interrupt so that it only reads data and enables main to process data • Created subroutine that will properly process data, in theory	• Data that was tested used simulated data, which means slave might not work • Data indicator now shows that it takes 8 seconds to process	• Fix slave code to read data properly • Create new code to process data correctly

5-1-26	• Added necessary files to Github • Added current issue to the README • Located where a few of my issues were	• Handshake was running every 2 seconds for an unknown reason • Code has several issues that I had run out of time to fix/determine	• Work on locating issues with code • Submit all work to project Github
--------	---	---	---

Appendix

Bloodhound Code Github Link (Earnest 2026):

<https://github.com/rc-earnest/BloodHound.git>

Universal Remote Controller Joystick/ Button Info (Christensen 2024):

https://docs.google.com/spreadsheets/d/11ozQTjpssO_NdbAwEpGcBfDm9fogVeNyMzyLpNkhu4g/edit?usp=sharing

Bloodhound BOM (Excluding Thermal Camera and Related Hardware):

<https://docs.google.com/spreadsheets/d/1DF1w89CaXs1tx9-P3O52QzfhAR6ybnsI/edit?usp=sharing&ouid=114733117654452189091&rtpof=true&sd=true>

DOCYKE S350 Servo Motor:

<https://www.docyke.com/products/servo-motor>

CONTRIBUTORS:

Zac Christensen

- Developed the Universal Remote Controller (URC) Board and Code
- Developed the foundational code for Master Main Board and Slave Tank Drive

Payden Hoskins

- Aided in creating functional and foundational subroutines for Motor Forward and Motor Reverse code within the Slave board.

Rudy Earnest

- Aided in editing the addresses for the Master Main Board code to allow for correct separation between the joysticks and buttons on the URC.

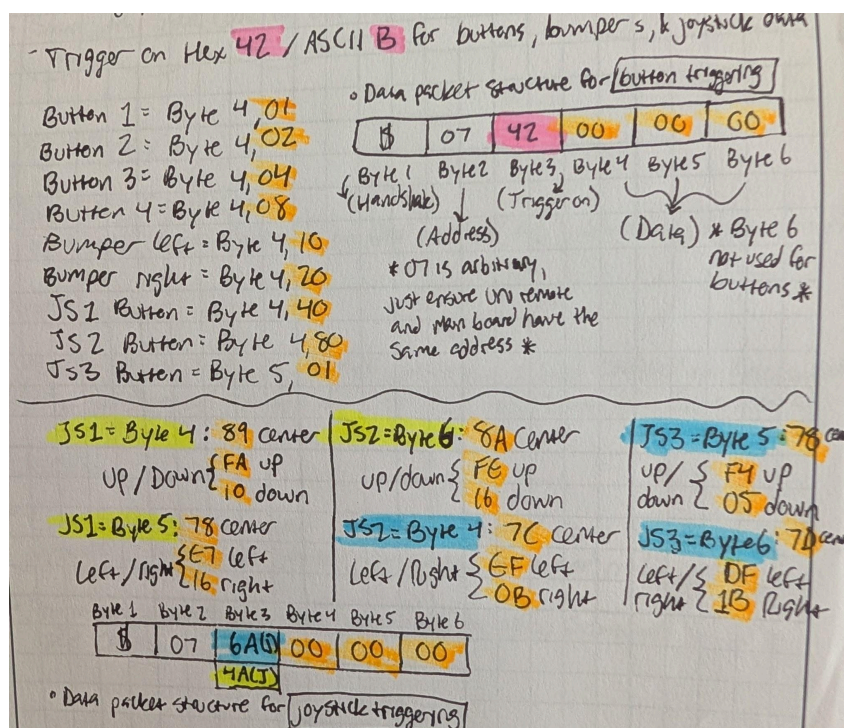


Figure 15: Joystick Data Packet Findings from March 18 Engineering Log

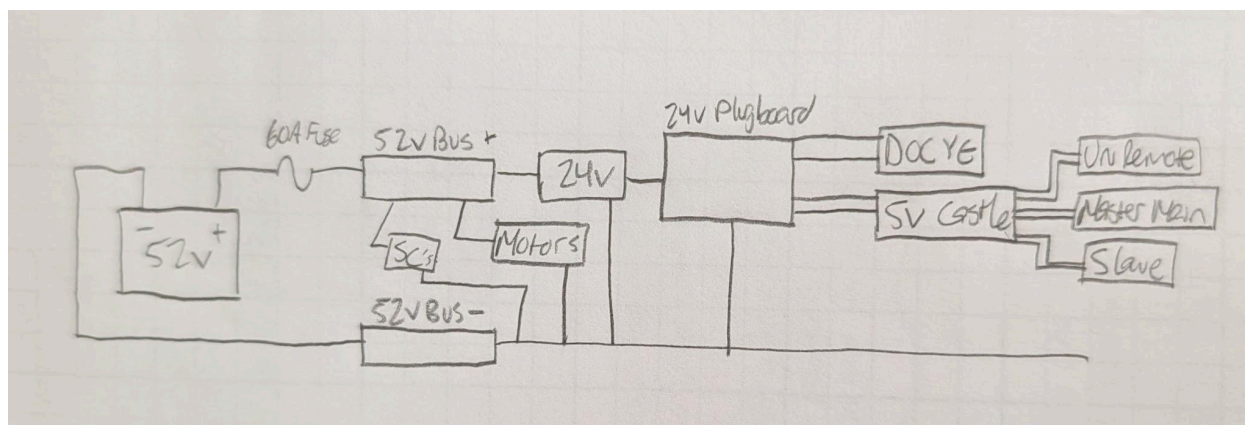


Figure 16: Wiring Diagram Sketch for Battery Power

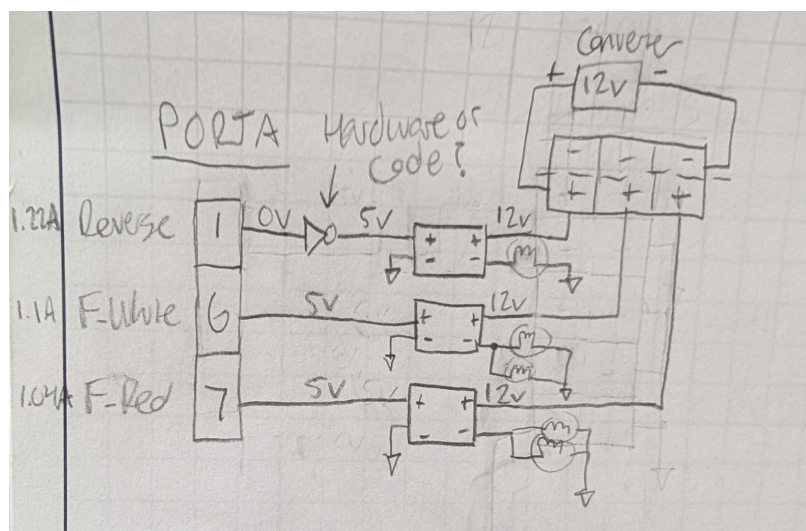


Figure 17: Wiring Diagram Sketch for Headlights & 12V Relays

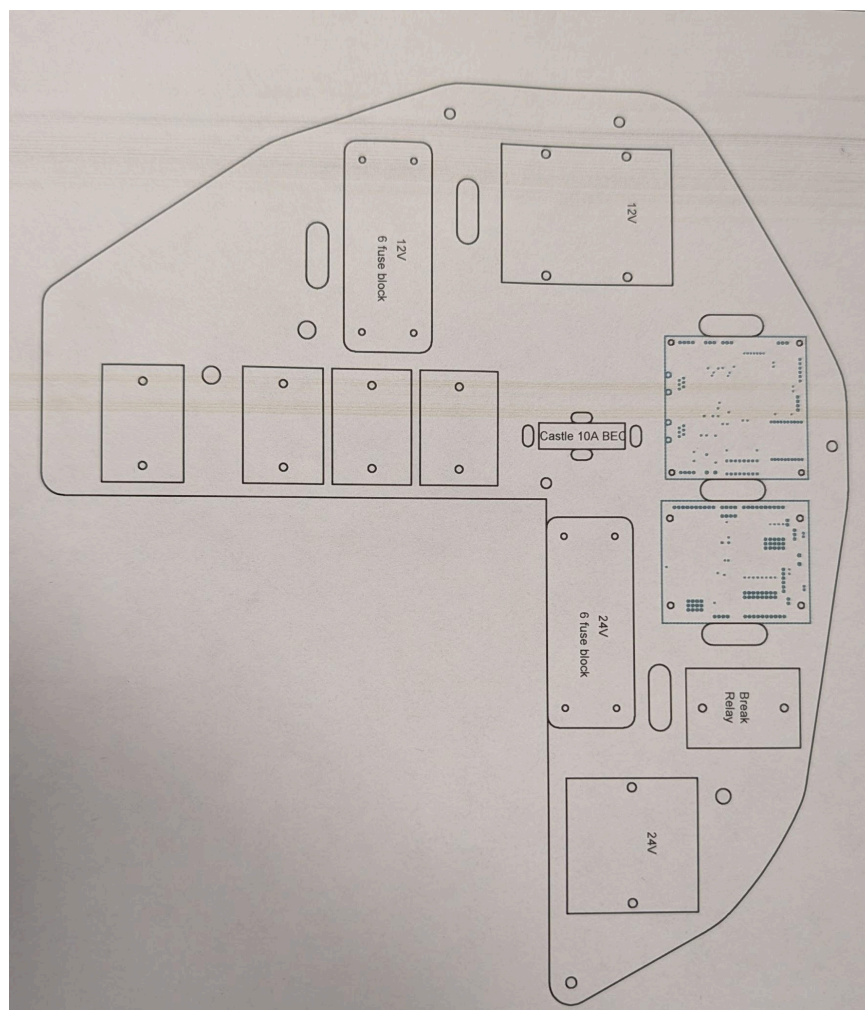


Figure 18: Hardware Mounting Panel Layout (Slack 2026)

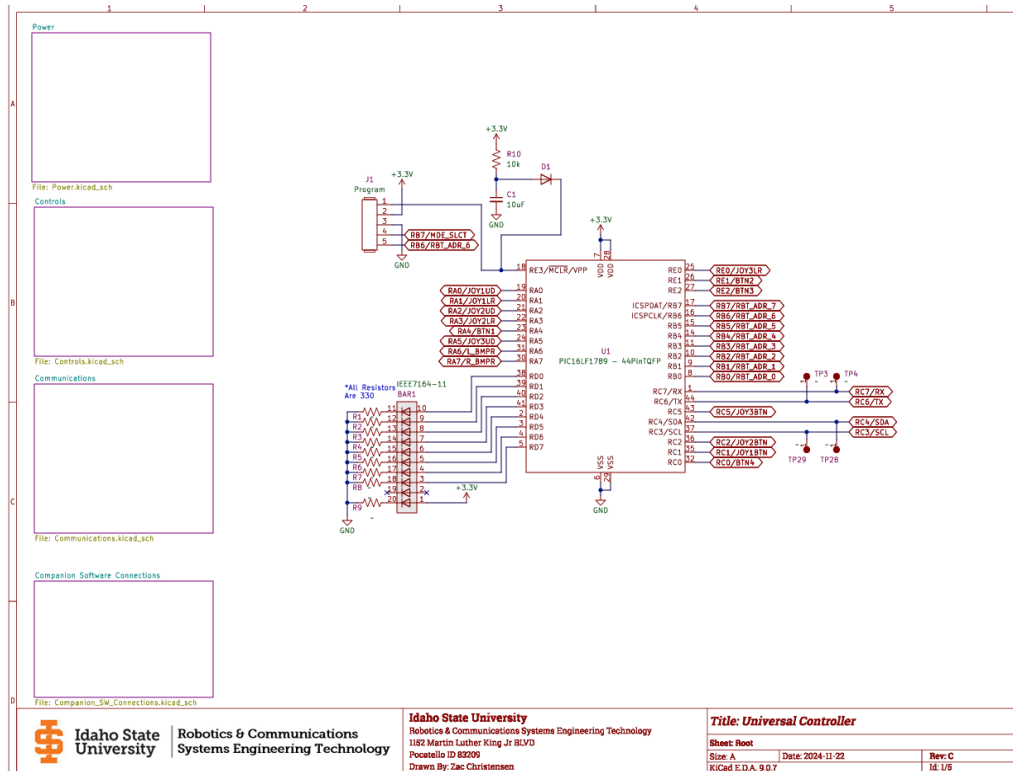


Figure 19: Universal Controller Schematic, Page 1 (Christensen 2024)

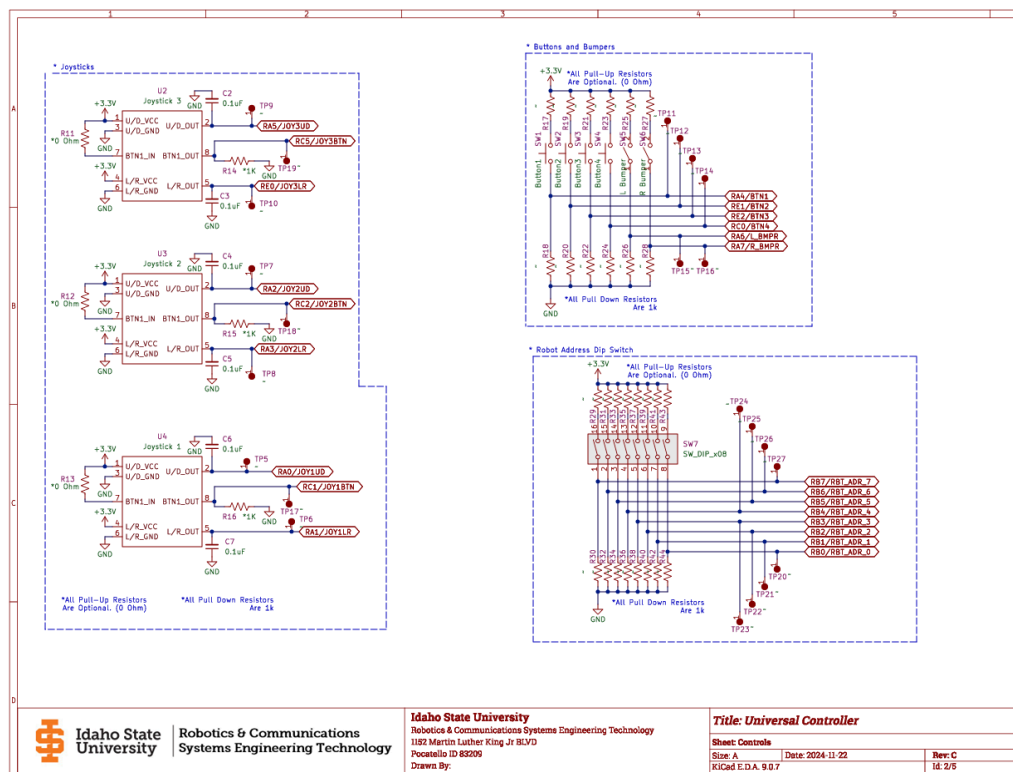


Figure 20: Universal Controller Schematic, Page 2 (Christensen 2024)

Figure 22: Universal Controller Schematic, Page 4 (Christensen 2024)

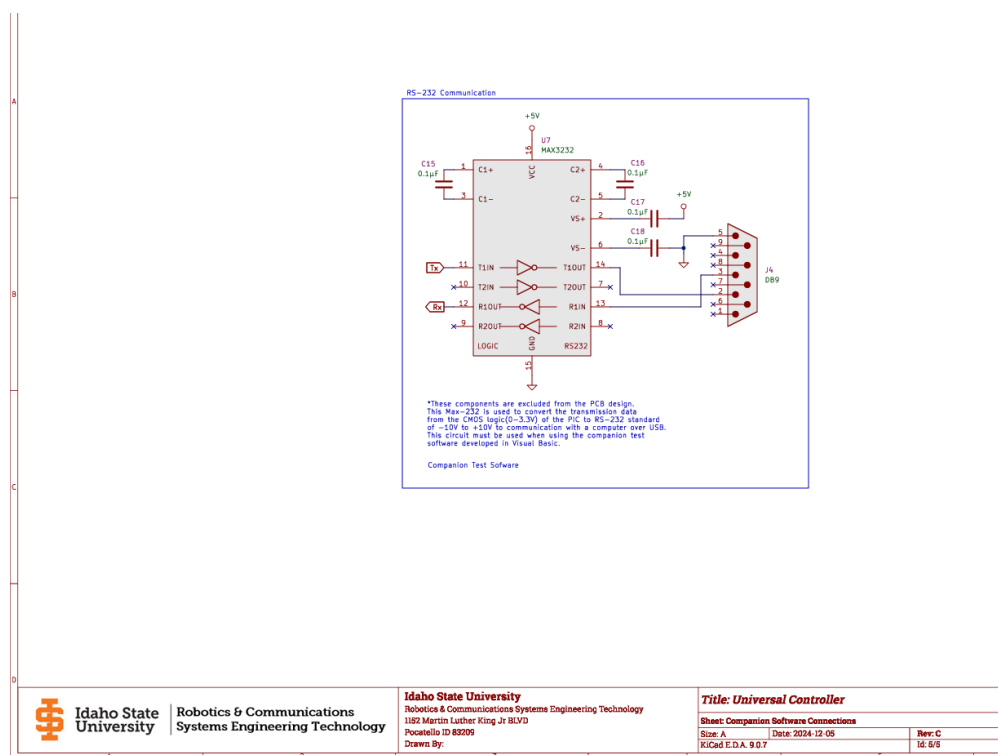


Figure 23: Universal Controller Schematic, Page 5 (Christensen 2024)

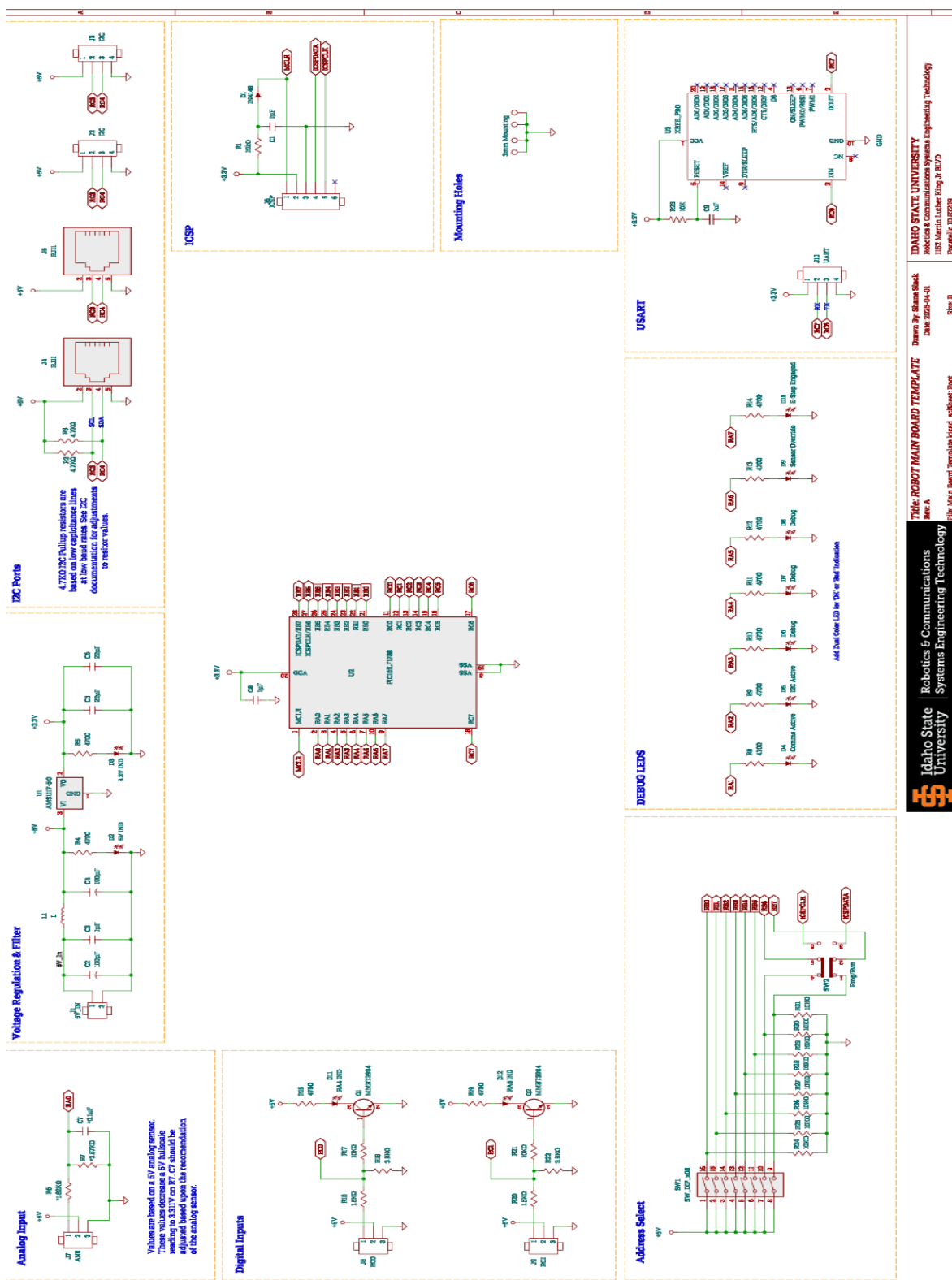


Figure 24: Universal Master/ Main Board Schematic (Slack 2025)

Figure 26: PIC Development/ Slave Board Schematic, Page 2 (Slack 2025)

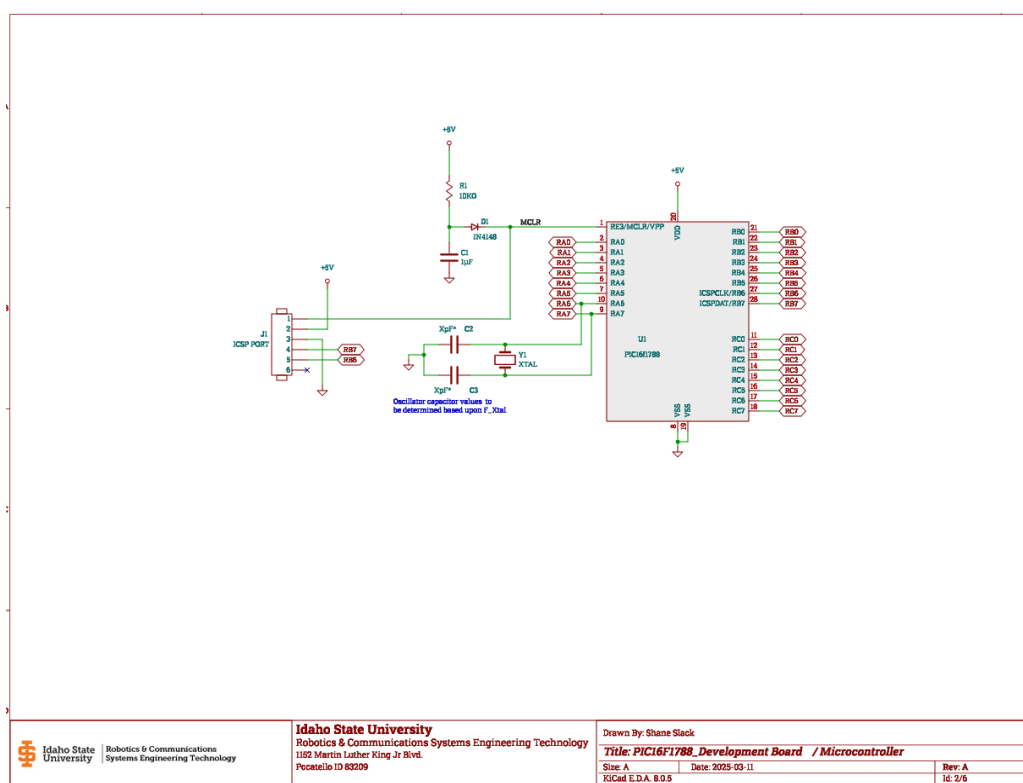


Figure 26: PIC Development/ Slave Board Schematic, Page 2 (Slack 2025)

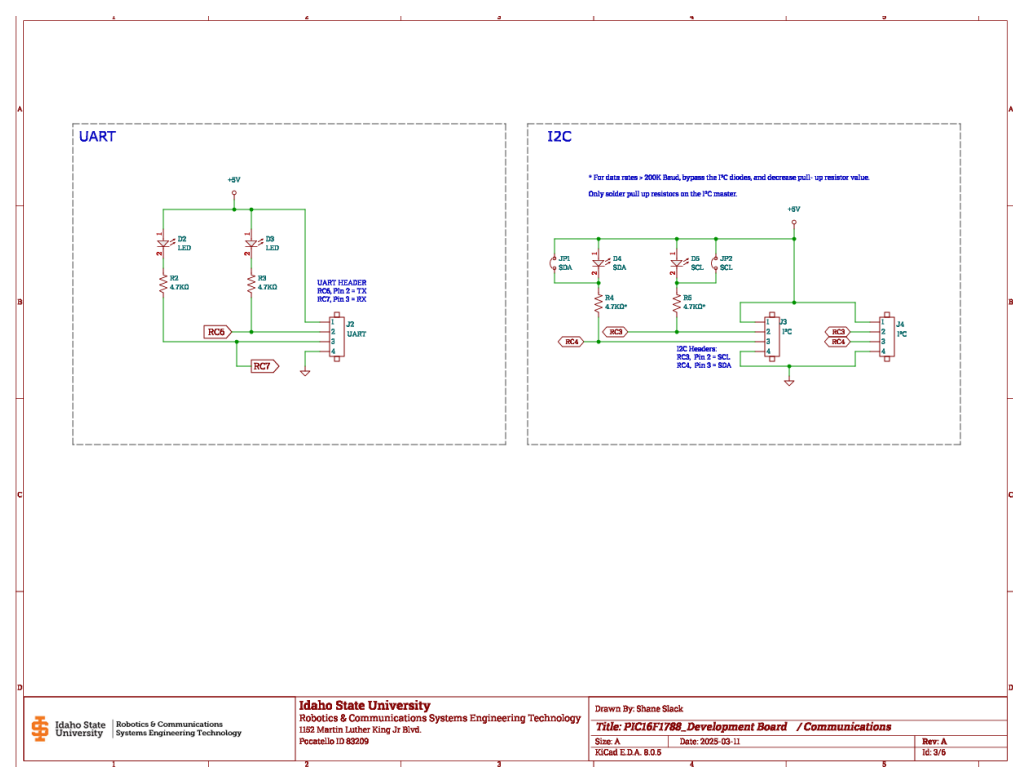


Figure 27: PIC Development/ Slave Board Schematic, Page 3 (Slack 2025)

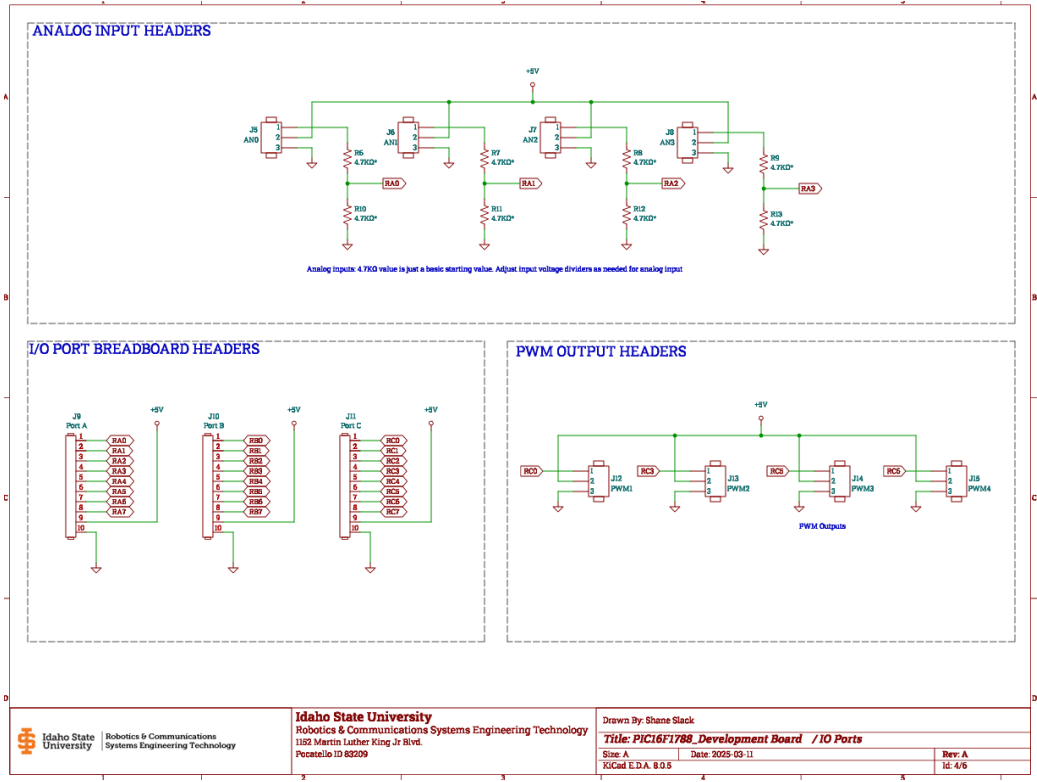


Figure 28: PIC Development/ Slave Board Schematic, Page 4 (Slack 2025)

Remove Debug jumpers when using PORTB for alternate functions

J73 Debug 7
J74 Debug 6
J75 Debug 5
J76 Debug 4
J77 Debug 3
J78 Debug 2
J79 Debug 1
J78 Debug 0

LED7
Debug 7
R32 4.7KΩ
LED6
Debug 6
R31 4.7KΩ
LED5
Debug 5
R30 4.7KΩ
LED4
Debug 4
R29 4.7KΩ
LED3
Debug 3
R28 4.7KΩ
LED2
Debug 2
R27 4.7KΩ
LED1
Debug 1
R26 4.7KΩ
LED0
Debug 0
R25 4.7KΩ

DEBUG LEDS

Figure 30: PIC Development/ Slave Board Schematic, Page 6 (Slack 2025)